

CS496 Quantum Computing Inline Diagrams

Exam-ready study guide built from the provided course slide files

Scope and source decision

Scope: I inspected the full uploaded zip. The Quantum Computing part is files _000 through _009: 10 PowerPoint decks, 184 slides total. Files _010 onward are Generative AI, LangChain, Agentic AI, and Blockchain/Solidity materials; they were inventoried but not summarized because they are outside the requested Quantum Computing topic.

Exam-important

This guide corrects one obvious slide mistake: in _003 slide 6, the text says CNOT flips the target when the first qubit is $|0\rangle$. The later truth table in _003 slide 9 is correct: CNOT flips the target only when the control qubit is $|1\rangle$.

Use this guide as the main revision file, but for a guaranteed top mark you should still know the lecture examples and be able to reproduce the formulas, circuits, and algorithms without looking.

2. _000 - Quantum & AGI Computing: Course Overview

This deck frames quantum computing as one part of the broader CS496 theme: AGI, Agentic AI, and blockchain. The quantum part begins with the difference between classical bits and qubits, then introduces superposition and entanglement.

Core ideas

- Classical bit: one definite value, 0 or 1.
- Qubit: quantum information unit that can be in a superposition of basis states before measurement.
- Superposition: a qubit can have amplitudes for both $|0\rangle$ and $|1\rangle$.
- Entanglement: multiple qubits can share a joint state whose information is not separable into independent qubit states.
- AGI context: the slides place quantum computing beside Agentic AI and blockchain as pillars in a future intelligent computing stack.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2, \quad |\alpha|^2 + |\beta|^2 = 1$$

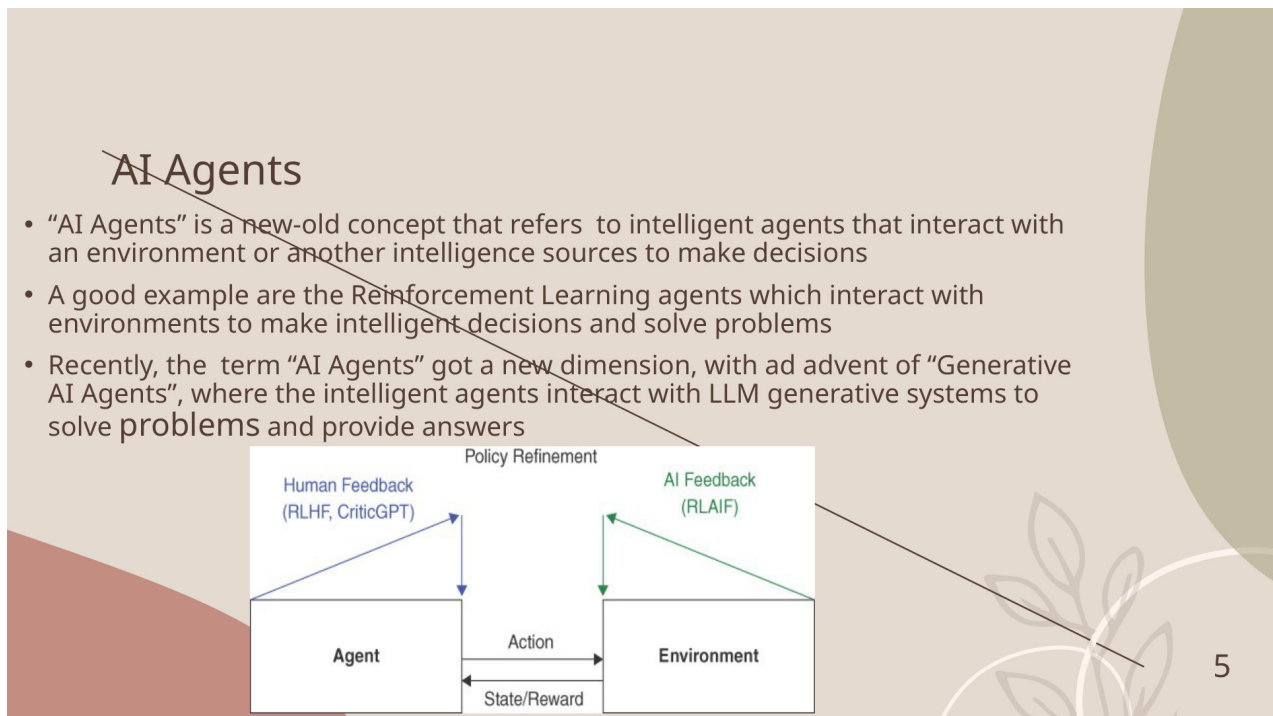


Diagram: AI-agent feedback loop from the course overview (_000 slide 5).

How to read it: The agent observes an environment, takes an action, receives feedback/reward, and updates behavior. This is not a quantum circuit, but it explains why the course connects quantum computing to autonomous intelligent systems.

Exam-important

Do not say quantum computers are fast just because they try all answers independently. The useful advantage comes from superposition plus controlled interference, entanglement, and measurement that amplifies useful answers and suppresses wrong ones.

3. _001 - Principles and Roots of Quantum Computing

Quantum mechanics roots

Quantum computing is grounded in the rules used to describe photons, electrons, atoms, and subatomic particles. The deck emphasizes that human-scale intuition is often misleading; the mathematics is the reliable guide.

Quantization, uncertainty, and coherence

- Quantization: certain physical quantities come in discrete units rather than arbitrary continuous values. Example: electrons occupy allowed energy/orbital states.
- Uncertainty: some pairs of quantities, such as position and momentum, cannot both be known exactly at the same time.
- Coherence: a quantum system maintains phase relationships that make superposition and interference possible. Noise, heat, vibration, and measurement can destroy coherence.

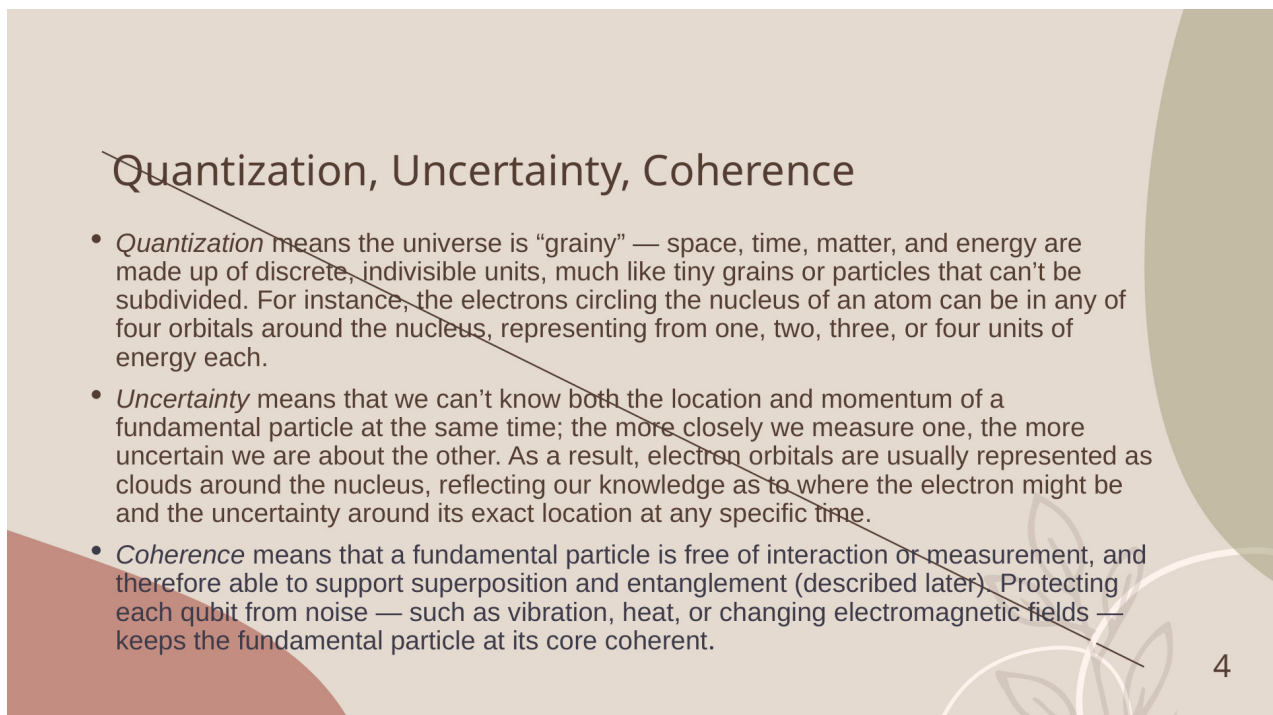


Diagram: Quantization, uncertainty, and coherence (_001 slide 4).

How to read it: Read this as the three physics requirements behind qubits: discrete state structure, limits on simultaneous measurement knowledge, and protected phase relationships.

Exam-important

Coherence is the condition that lets quantum behavior stay usable. Decoherence is the loss of that behavior due to interaction with the environment; it is one of the biggest practical barriers in quantum hardware.

Wavefunction and probabilities

The wavefunction describes the quantum state. Its amplitudes are not probabilities directly; probabilities come from squared magnitudes of amplitudes.

Single qubit state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Normalization:

$$|\alpha|^2 + |\beta|^2 = 1$$

Measurement probabilities:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2$$

Entanglement and interference

Entanglement means a multi-particle state cannot be fully described as separate individual states. Interference means amplitudes combine: matching phases reinforce each other and opposite phases cancel. Quantum algorithms are designed so the desired answer gets constructive interference and wrong answers get destructive interference.

Quantum Interference

Quantum interference is a fundamental phenomenon in quantum mechanics where the probability amplitudes of quantum states combine—either constructively or destructively—to influence the likelihood of outcomes. Unlike classical interference, which deals with overlapping physical waves (like sound or water), quantum interference arises from the wave-like behavior of particles such as electrons, photons, and even atoms.

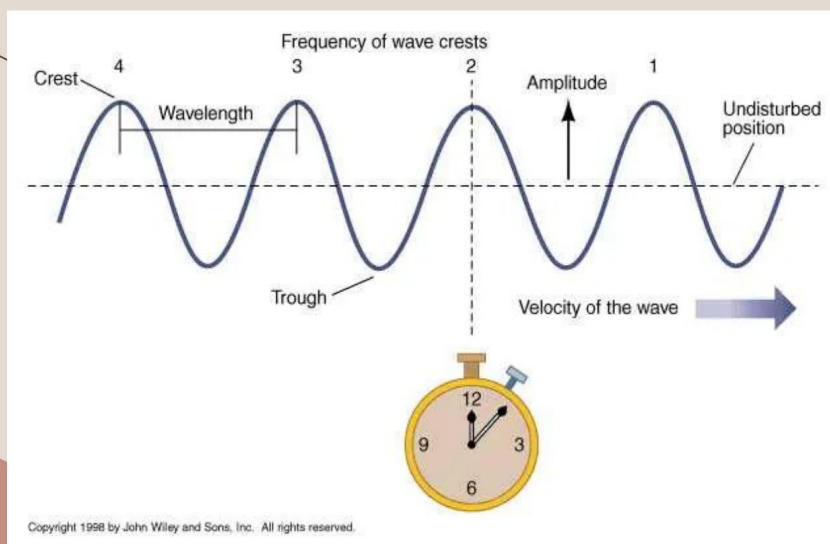
In quantum theory, particles are described by wavefunctions, which contain complex-valued probability amplitudes. When a quantum system can evolve along multiple possible paths—such as going through two slits in a barrier—each path has an associated amplitude. These amplitudes can interfere when the paths are indistinguishable, meaning we cannot tell which path the particle actually took.

10

Diagram: Quantum interference explanation (_001 slide 10).

How to read it: Focus on amplitudes, not probabilities. Amplitudes can be positive, negative, or complex; when paths combine, they can reinforce or cancel. This is why phase gates matter.

Wave Parameters



14

Diagram: Wave parameters diagram (_001 slide 14).

How to read it: Amplitude is height, wavelength is horizontal spacing, frequency is cycles per unit time, and phase is horizontal shift. Phase is exam-important because quantum interference depends on relative phase.

Historical milestones

The slides give a timeline from early quantum theory to quantum algorithms and modern hardware. The most exam-relevant milestones are Planck quantization, quantum mechanics in the 1920s, EPR entanglement, Turing computation, Feynman quantum simulation, Deutsch quantum Turing machines, Shor factoring, Grover search, quantum error correction, and recent quantum supremacy / scaling work.

IBM Eagle Quantum Chip

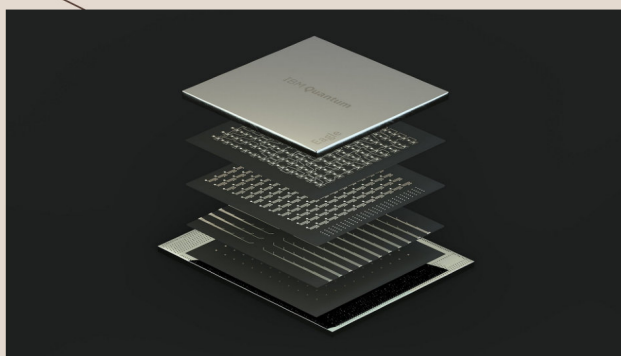


21

Diagram: IBM Eagle quantum chip image (_001 slide 21).

How to read it: The hardware image is a reminder that qubits are physical systems. Real devices must isolate qubits from noise while still allowing controlled gates and measurement.

IBM Quantum Chip: Eagle interior -127 Qubits



22

Diagram: IBM Eagle chip internal layered view (_001 slide 22).

How to read it: This visualization highlights the engineering side: control, connectivity, and scaling are as important as the theory.

Exam-important

Feynman's key motivation: classical computers are inefficient for simulating quantum systems. A quantum computer naturally represents quantum states and can simulate them more directly.

4. _002 - Quantum Gates and Operators

Why gates matter

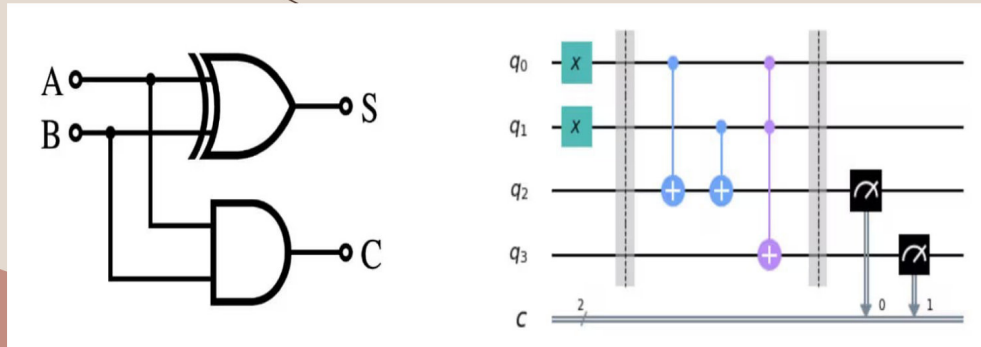
Quantum algorithms are built as circuits. Gates are operators applied to qubit states. Most quantum gates are unitary, which means they are reversible and preserve total probability. Measurement is different: it collapses the quantum state into a classical outcome.

Operator	Gate(s)	Matrix
Pauli-X (X)	 	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)		$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)		$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)		$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$ (T)		$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)	 	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP	 	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli (CCNOT, CCX, TOFF)		$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

Diagram: Operator/circuit/matrix table (_002 slide 3).

How to read it: Each row links a gate name to its circuit symbol and matrix. For exams, know both the effect on basis states and the matrix for H, X, Y, Z, phase, SWAP, CNOT, Toffoli, and Fredkin.

Two examples of Circuits: Classical VS Quantum



7

Diagram: Classical circuit vs quantum circuit example (_002 slide 7).

How to read it: The left circuit uses classical irreversible logic; the right circuit uses quantum wires and gates. Quantum circuit lines represent qubits evolving over time, not physical wires carrying a definite bit value.

Bloch sphere and one-qubit states

The Bloch sphere represents any single-qubit pure state using two angles. The north pole is $|0\rangle$, the south pole is $|1\rangle$, and points on the equator are equal superpositions with different phases.

$$|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$$

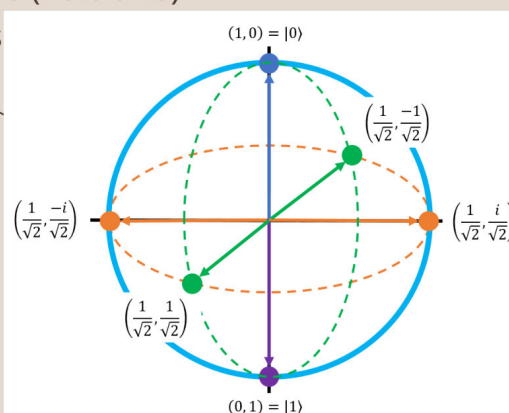
The Bloch Sphere

- This nice simple figure on the right is Bloch Sphere
- The Bloch Sphere is unitary figure used to represent the state of a qubit

The vertical line in the middle is the Z axis (Pole axis)

The green line in the middle is the X axis

The orange arrow is the Y axis

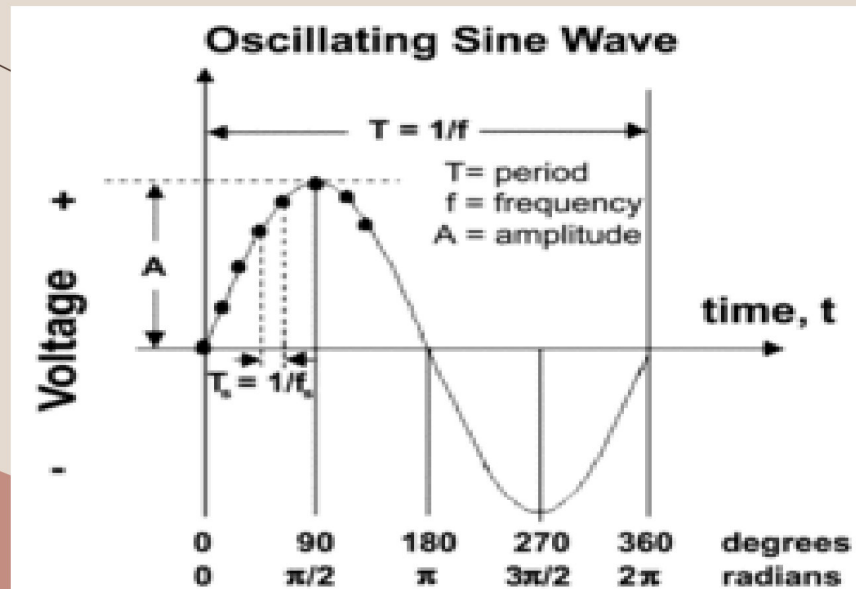


8

Diagram: Bloch sphere diagram (_002 slide 8).

How to read it: θ moves the state between $|0\rangle$ and $|1\rangle$; ϕ changes phase around the equator. A gate is visualized as a rotation or reflection of the state vector.

Some Concepts Regarding Waves



11

Diagram: Sine wave and radians refresher (_002 slide 11).

How to read it: Quantum gates are often rotations, so radians and phase are not decoration. A full cycle is 2π radians; π is half a cycle; $\pi/2$ is a quarter cycle.

Exam-important

The Bloch sphere is only a direct picture for one qubit. Multi-qubit states live in larger vector spaces and cannot usually be drawn as one simple sphere.

Single-qubit gates

The main single-qubit gates transform amplitudes and phases. Some change measurement probabilities; others only change phase, which still matters because later interference depends on phase.

The Hadamard Gate

- The Hadamard gate is a π rotation around an axis halfway between the x and z axes on the Bloch sphere.
- Applying the H gate to $|0\rangle$ creates a superposition state such as $\frac{|0\rangle + |1\rangle}{\sqrt{2}}$. The matrix representation of the H gate is below.

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

13

Diagram: Hadamard gate definition (_002 slide 13).

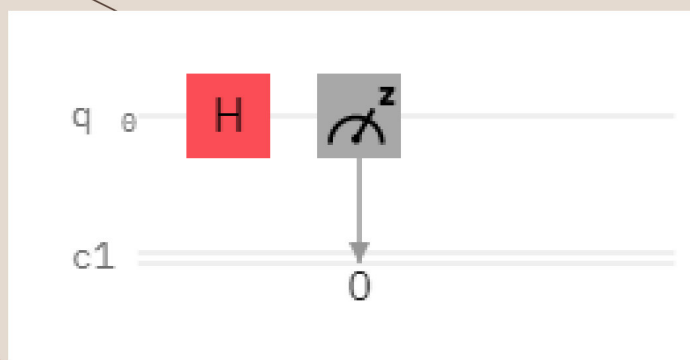
How to read it: Hadamard sends a basis state to an equal superposition. It is usually the first gate applied when an algorithm needs to explore many basis states.

$$H = (1/\sqrt{2}) \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2} = |+\rangle$$

$$H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle$$

The Hadamard Gate Circuit



16

Diagram: Hadamard circuit and measurement (_002 slide 16).

How to read it: Start with $|0\rangle$, apply H, then measurement gives 0 or 1 with equal probability. The gate creates superposition; measurement samples from it.

The Hamdard Gate in Action

```
from qiskit import QuantumCircuit, transpile, ClassicalRegister,
    QuantumRegister
from qiskit_aer import AerSimulator
backend = AerSimulator() # Using local Aer simulator

q = QuantumRegister(1,'q') # Initialise quantum register
c = ClassicalRegister(1,'c') # Initialise classical register
circuit = QuantumCircuit(q,c) # Initialise circuit
circuit.h(q[0]) # Put Qubit 0 in to superposition using hadamard gate
circuit.measure(q,c) # Measure qubit
circ = transpile(circuit, backend) # Rewrites the circuit to match the
    backend's basis gates and coupling map
shots = 1024 # Run and get counts

result = backend.run(circ,shots=shots).result()
counts = result.get_counts(circ)
print('RESULT: ',counts) # Print result
```

17

Diagram: Hadamard code example (_002 slide 17).

How to read it: The code builds a 1-qubit circuit, applies H, measures, and counts outcomes. The roughly 50/50 counts verify equal superposition.

The H Gate

Computing $H|0\rangle$

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix} = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

- This superposition state is so common and important, that it is given its own symbol:

$$|+\rangle \equiv \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

- By applying the H gate on the $|0\rangle$, we created a superposition of $|0\rangle$ and $|1\rangle$ where measurement in the computational basis (along z in the Bloch sphere picture) would give you each state with equal probabilities.

The $|-\rangle$ state

You might have guessed that there is a corresponding $|-\rangle$ state:

$$|-\rangle \equiv \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

18

Diagram: Hadamard matrix computation (_002 slide 18).

How to read it: Read the multiplication as the mathematical version of the circuit: the H matrix transforms the state vector for $|0\rangle$ into equal amplitudes for $|0\rangle$ and $|1\rangle$.

The X Gate (NOT Gate)

- The X gate is a π rotation around the x axis of the Bloch sphere.
- Applying the X gate to $|0\rangle$ results in $|1\rangle$, and applying the X gate to $|1\rangle$ results in $|0\rangle$, so it is an operation similar to the classical NOT gate, and is also known as bit flip.
- The matrix representation of the X gate is below.

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

20

Diagram: X gate / NOT gate (_002 slide 20).

How to read it: X flips computational basis states: $|0\rangle$ becomes $|1\rangle$, and $|1\rangle$ becomes $|0\rangle$. On the Bloch sphere it is a 180-degree rotation around the x -axis.

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$
$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

Pauli-Y Gate

Similarly to Pauli-X gate, the Y gate represents a rotation of around the y axis by π radians.

It transforms $|0\rangle$ to $i|1\rangle$ and $|1\rangle$ to $-i|0\rangle$.

$$\text{---}[Y]\text{---} = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} = i|1\rangle\langle 0| - i|0\rangle\langle 1|$$

24

Prepared By: Iqra Naz

Diagram: Pauli-Y gate (_002 slide 24).

How to read it: Y flips the basis state and adds complex phase factors. This matters in phase-sensitive algorithms and rotations.

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$
$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle$$

Pauli-Z Gate

The Z gate is actually a special case of the phase shift gate where $\phi = \pi = 180^\circ$.

It represents a rotation around the z axis by π radians.

It has no effect on $|0\rangle$ but transforms $|1\rangle$ to $-|1\rangle$.

$$Z|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

Prepared By: Iqra Naz

25

Diagram: Pauli-Z gate (_002 slide 25).

How to read it: Z does not flip $|0\rangle$ or $|1\rangle$; it flips the phase of $|1\rangle$. You may not see its effect immediately in a direct measurement, but it changes interference later.

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

Phase Shifter Gate

- This is a family of single-qubit gates that leave the basis state $|0\rangle$ unchanged and map $|1\rangle$ to $e^{i\phi}|1\rangle$.
- The probability of measuring a $|0\rangle$ or $|1\rangle$ is unchanged after applying this gate, however it modifies the phase of the quantum state.
- This is equivalent to tracing a horizontal circle (a line of latitude) on the Bloch sphere by ϕ radians.

$$|x\rangle \xrightarrow{F_\theta} e^{i\theta}|x\rangle \quad F_\theta = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}$$

27

Diagram: Phase shifter gate (_002 slide 27).

How to read it: The phase gate keeps $|0\rangle$ unchanged and multiplies $|1\rangle$ by $e^{i\phi}$. Measurement probabilities are unchanged immediately, but downstream interference changes.

$$P(\phi) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{bmatrix}$$

S and S[†] Gates

The Phase gate (S gate) is a single-qubit operation.

The S gate is also known as the phase gate or the Z90 gate, because it represents a 90-degree rotation around the z-axis.

The conjugate transpose of S gate is S[†] gate.

$$S = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{2}} \end{bmatrix}, \quad S^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-i\frac{\pi}{2}} \end{bmatrix}$$

Prepared By: Iqra Naz

28

Diagram: S and S-dagger gates (_002 slide 28).

How to read it: S is a phase gate with $\phi = \pi/2$. S[†] is its inverse. Adjoins undo the corresponding phase rotation.

T and T[†] Gates

The T-gate is a very commonly used gate, it is an R_φ-gate with $\phi = \pi/4$.

As with the S-gate, the T-gate is sometimes also known as (Z)^(1/4) gate.

The conjugate transpose of T gate is T[†] gate.

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix}, \quad T^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-i\frac{\pi}{4}} \end{bmatrix}$$

Prepared By: Iqra Naz

29

Diagram: T and T-dagger gates (_002 slide 29).

How to read it: T is a phase gate with $\phi = \pi/4$. T and Clifford gates are important in universal fault-tolerant gate sets.

General U gates

- Z, S & T-gates were all special cases of the more general R_ϕ -gate.
- The U_3 -gate is the most general of all single-qubit quantum gates.
- It is a parameterized gate of the form:

$$U_3(\theta, \phi, \lambda) = \begin{bmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ e^{i\phi} \sin(\theta/2) & e^{i\lambda+i\phi} \cos(\theta/2) \end{bmatrix}$$

$$U_3(\frac{\pi}{2}, \phi, \lambda) = U_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -e^{i\lambda} \\ e^{i\phi} & e^{i\lambda+i\phi} \end{bmatrix} \quad U_3(0, 0, \lambda) = U_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\lambda} \end{bmatrix}$$

30

Diagram: General U gate (_002 slide 30).

How to read it: $U(\theta, \phi, \lambda)$ is a general parameterized one-qubit unitary. Many circuit libraries compile arbitrary one-qubit rotations into U-like gates.

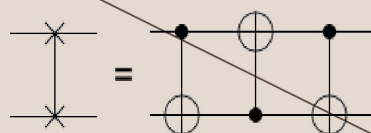
Multi-qubit gates

Multi-qubit gates create relationships between qubits. CNOT, controlled phase gates, Toffoli, and Fredkin are central because they implement conditional logic and can produce entanglement.

SWAP Gate

- The swap gate swaps two qubits.
- With respect to the basis $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$.
- It is represented by the matrix:

$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



32

Diagram: SWAP gate (_002 slide 32).

How to read it: SWAP exchanges two qubit states. The matrix leaves $|00\rangle$ and $|11\rangle$ unchanged but swaps $|01\rangle$ and $|10\rangle$.

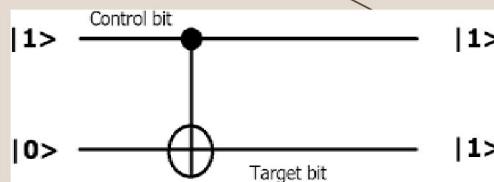
Controlled Gates

Controlled gates require at least one control and one target qubit.

The gate in question will only operate on the target qubit if the control qubit is in a specific state.

Types of control gates:

- Cx Gate
- Cy Gate
- Cz Gate



Prepared By: Iqra Naz

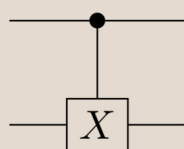
33

Diagram: Controlled gates overview (_002 slide 33).

How to read it: Controlled gates have a control and a target. The operation is applied to the target only when the control condition is satisfied, normally control = $|1\rangle$.

Cx Gate

- Cx leaves the control qubit unchanged and performs a Pauli-X gate on the target qubit when the control qubit is in state $|1\rangle$, leaves the target qubit unchanged when the control qubit is in state $|0\rangle$.



$$CX \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

34

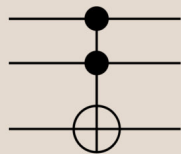
Diagram: CX / CNOT gate (_002 slide 34).

How to read it: The control qubit passes unchanged. The target is X-flipped if and only if the control is $|1\rangle$. Memorize the truth table.

$CNOT|00\rangle = |00\rangle$
 $CNOT|01\rangle = |01\rangle$
 $CNOT|10\rangle = |11\rangle$
 $CNOT|11\rangle = |10\rangle$

Toffoli Gate

- The Toffoli gate is a three qubit gate with two controls and one target.
- It performs an X on the target only if both controls are in the state $|1\rangle$.
- The final state of the target is then equal to either the AND or the NAND of the two controls, depending on whether the initial state of the target was $|0\rangle$ or $|1\rangle$.
- A Toffoli can also be thought of as a controlled-controlled-NOT, and is also called the CCX gate.



1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1

37

Diagram: Toffoli / CCNOT gate (_002 slide 37).

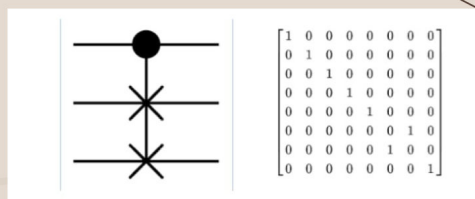
How to read it: Toffoli has two controls and one target. It flips the target only when both controls are $|1\rangle$, so it implements reversible AND-style logic.

Fredkin Gate

It is controlled-SWAP gate.

It only swaps the values of the second and third bits only if the first bit is set to 1.

It is also a reversible gate.



Prepared By: Iqra Naz

38

Diagram: Fredkin / CSWAP gate (_002 slide 38).

How to read it: Fredkin swaps two target qubits only if the control is $|1\rangle$. Like Toffoli, it is reversible and useful for conditional data movement.

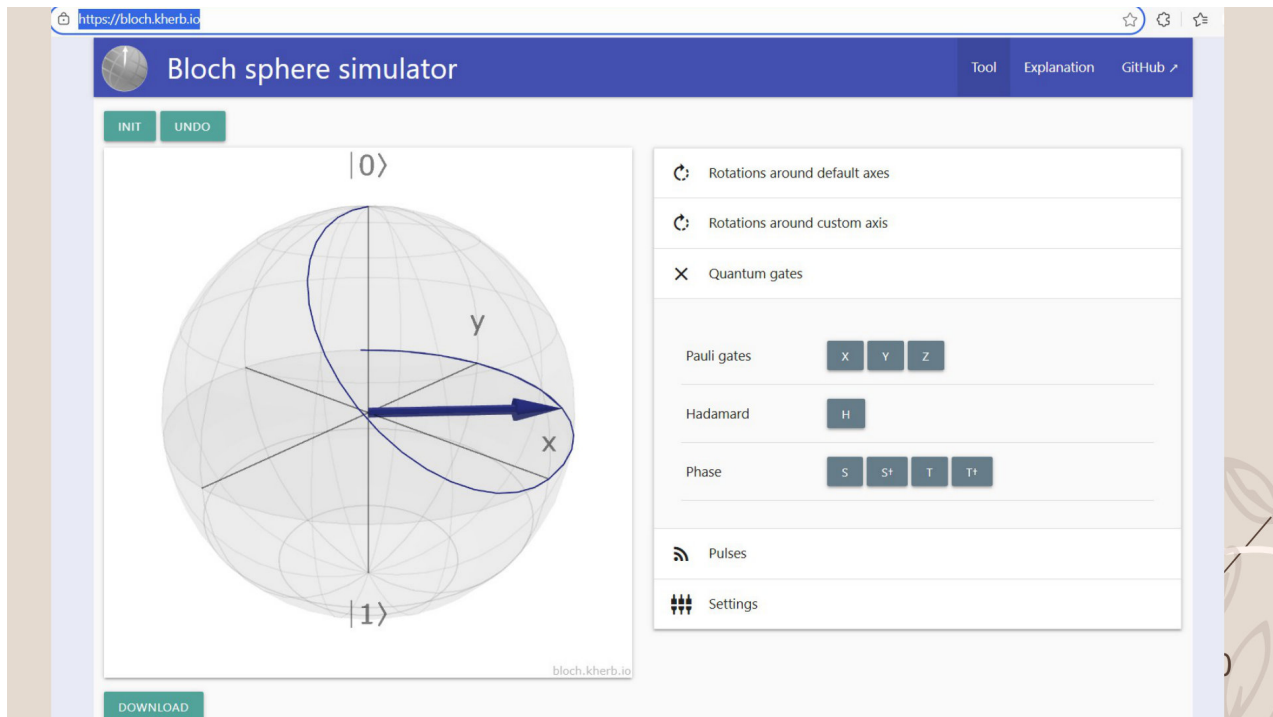


Diagram: Bloch sphere visualization tool (_002 slide 40).

How to read it: Use simulators to connect gate matrices to state-vector movement. For the exam, the mental model should match the matrix action.

Exam-important

High-yield gate facts: H creates superposition, X flips bits, Z flips phase, phase/S/T change relative phase, CNOT entangles when the control is in superposition, Toffoli provides reversible AND/carry behavior.

5. _003 - Entanglement, Bell States, and Multi-Qubit Systems

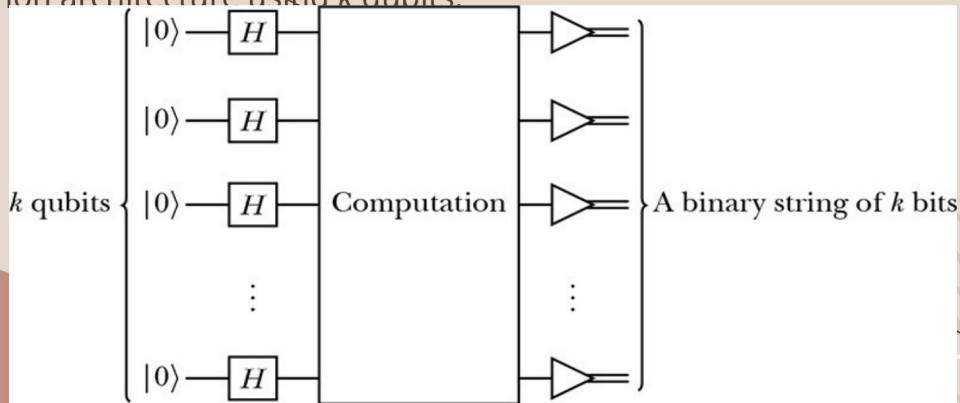
Initializing multiple qubits

Many algorithms start with qubits in $|0\rangle$ and apply H gates to create a uniform superposition across all basis states. For n qubits, applying H to every qubit creates amplitudes over 2^n computational basis states.

$$H^{\otimes n} |0 \dots 0\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$$

Most Algorithms Initialize qubits to $|0\rangle$ then go into H

As a result, many quantum algorithms begin with a bunch of qubits initialized to $|0\rangle$, which immediately go into H gates. Then we do some computing and finally measure the qubits. The Figure Below shows this common architecture using k qubits.



4

Diagram: Multiple qubits initialized with Hadamards (_003 slide 4).

How to read it: Each H gate creates a superposition on its qubit. Together, the register can represent all bit strings simultaneously as amplitudes.

Tensor products and product states

Multi-qubit states are built with tensor products. A product state can be separated into individual qubit states. An entangled state cannot be separated this way.

$$|00\rangle = |0\rangle \otimes |0\rangle$$

$$|0+\rangle = |0\rangle \otimes (|0\rangle + |1\rangle)/\sqrt{2}$$

Important Note: Two-qubit states are tensor products of the individual qubit states

- The tensor product of two one-qubit states always yields a two-qubit state.
- Such states, which can be decomposed as the tensor product of other states, are said to be product states. For example, $|00\rangle$ is a product state because it's just a shortened way of writing $|0\rangle \otimes |0\rangle$.
- Likewise, the following state is also a product state

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

is also a product state, because it is equal to $|+\rangle|+\rangle$.

Similarly, the state below is a product state $\frac{1}{\sqrt{2}}(|00\rangle + |01\rangle)$, which is the same state as $|0\rangle|+\rangle$.

One Exception is the Entanglement State: cannot be obtained as a product state

8

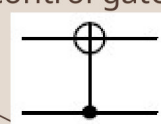
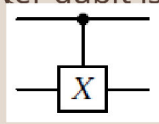
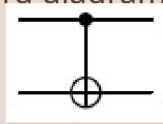
Diagram: Tensor products and product states (_003 slide 8).

How to read it: The slide tests whether a two-qubit state can be factored into two one-qubit states. If it cannot be factored, it is entangled.

CNOT and entanglement

The CNOT Gate (The XNOT gate)

- As we are going to see in a while, Entanglement is implemented using the two-qubit
- In the first two diagrams, the first qubit in the control gate, while the lower is the target gate
- In the third diagram, the lower qubit is the control gate, while the upper is the target



- We have seen the unitary representation of the X gate before
- Here the CNOT gate flips the output of the target qubit if the state of the first qubit is $|0\rangle$, otherwise it leaves the state of the target qubit s is

6

Diagram: CNOT gate drawing and correction note (_003 slide 6).

How to read it: Use the circuit symbol, but correct the text: CNOT flips the target when the control is $|1\rangle$, not $|0\rangle$. The later truth table confirms this.

The CNOT Gate

The CNOT, controlled-NOT, or controlled-X gate is a two-qubit gate that **cannot be obtained** as the tensor product of two one-qubit gates.

Its name is derived from the way it acts on the computational basis, which is the following:

$$\text{CNOT } |00\rangle = |00\rangle,$$

$$\text{CNOT } |01\rangle = |01\rangle,$$

$$\text{CNOT } |10\rangle = |11\rangle,$$

$$\text{CNOT } |11\rangle = |10\rangle.$$

As can be observed, the value of the second qubit is reversed (or negated) if and only if the value of the first qubit is 1.

In this way, we can say that the first qubit controls whether an X (also called NOT) gate is applied to the second qubit or not, hence the name of the operation.

Usually, the first qubit is called the control qubit and the second one is called the target qubit. The action on non-basis states is obtained by linearity. Indeed, we have that

9

Diagram: CNOT truth table (_003 slide 9).

How to read it: The second qubit is reversed only when the first qubit is 1. This is the authoritative CNOT rule for the guide.

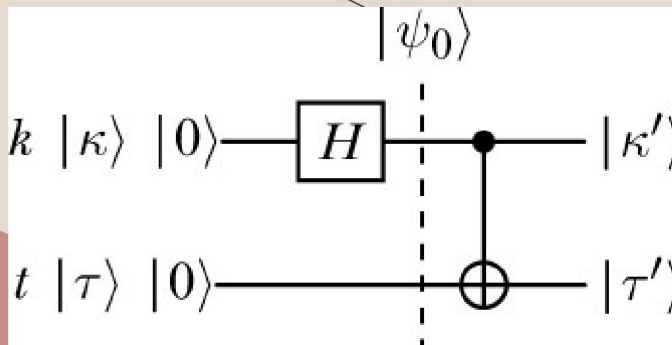
The standard two-qubit entanglement circuit is H on the first qubit followed by CNOT from the first qubit to the second.

$$|00\rangle \xrightarrow{\text{H on } q_0} (|00\rangle + |10\rangle)/\sqrt{2}$$

$$\xrightarrow{\text{CNOT } q_0 \rightarrow q_1} (|00\rangle + |11\rangle)/\sqrt{2}$$

Circuit Implementation of Entanglement

- As apparent from the diagram below, Entanglement is implemented using the Hadamard gate, followed by the cross-qubit CNOT Gate
- The Hadamard gate puts the first qubit in superposition mode, while the CNOT gate controls the output of the second qubit to provide a shared entanglement state



7

Diagram: Circuit implementation of entanglement (_003 slide 7).

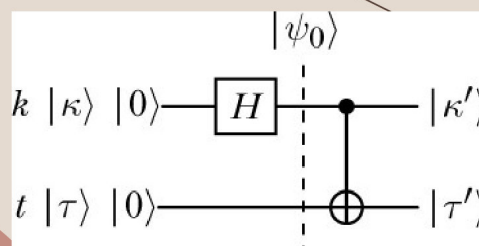
How to read it: H puts the first qubit in superposition. CNOT copies correlation, not a copy of the unknown state, onto the second qubit. The output is a shared entangled state.

Compute the components of the input state $|+0\rangle$ at $|\psi_0\rangle$

- Compute the input state $|+0\rangle$ at $|\psi_0\rangle$
noting that $V = 1/\sqrt{2}$

We compute the state at point $|\psi_0\rangle$ in the diagram below:

$$|\psi_0\rangle = |+0\rangle = |+\rangle \otimes |0\rangle = V \begin{bmatrix} 1 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = V \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$



10

Diagram: Input state $|+0\rangle$ before CNOT (_003 slide 10).

How to read it: The first qubit has amplitudes for $|0\rangle$ and $|1\rangle$; the second starts at $|0\rangle$. This is the input to the CNOT transformation.

Compute the CNOT transformation

- Compute the CNOT transformation:

$$v = 1/\sqrt{2}$$

$$CX|+0\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \left(v \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right) = v \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |\gamma\rangle$$

11

Diagram: CNOT transformation calculation (_003 slide 11).

How to read it: Matrix multiplication maps $|00\rangle$ to $|00\rangle$ and $|10\rangle$ to $|11\rangle$, producing the entangled Bell state.

Bell states

Bell states are four maximally entangled two-qubit states. They are core examples for teleportation and quantum communication.

$$|\Phi^{++}\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$$

$$|\Phi^{--}\rangle = (|00\rangle - |11\rangle)/\sqrt{2}$$

$$|\Psi^{++}\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$$

$$|\Psi^{--}\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$$

The Bell States: Perfect Entanglement

- **Bell states** are a specific type of entangled state that are maximally entangled, meaning that the information shared between the two qubits is perfectly correlated.
- This unique property is crucial for many areas in quantum physics, especially in quantum information science.
- In simple terms, if one qubit of a Bell state is measured, the other qubit will immediately show a corresponding result, no matter how far apart they are.
- This relationship is what makes Bell states quite fascinating and a key resource for various quantum technologies. The Bell states are a form of entangled and normalized basis vectors, which implies that the overall probability of the particles being in one of the mentioned states is 1.
- This entanglement means that the measurement outcomes are correlated, and the state of one particle is dependent on the state of the other particles.
- Understanding the relationship between Bell states and entanglement is useful in analysis of quantum communication, such as superdense coding and quantum teleportation. These mechanisms cannot transmit information faster than the speed of light, a result known as the no-communication theorem.

13

Diagram: Bell states as perfect entanglement (_003 slide 13).

How to read it: Bell states are maximally entangled: measuring one qubit immediately determines the correlated value of the other. This correlation does not transmit usable information faster than light.

Bell States

- Bell states are the four states that can be created when two qubits are maximally entangled. The four states are represented as follows:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

$$|\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$$

$$|\Psi^+\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}}$$

$$|\Psi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}$$

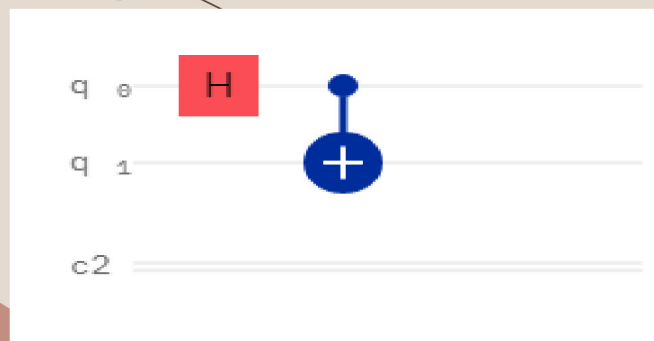
14

Diagram: Four Bell states table (_003 slide 14).

How to read it: Memorize the four states by whether they use 00/11 or 01/10 and whether the sign is plus or minus.

The First Bell State

- The first Bell state is incredibly easy to implement as it can be created using a two qubit circuit consisting of a Hadamard gate and CNOT gate found below:



15

Diagram: First Bell state circuit (_003 slide 15).

How to read it: The circuit H then CNOT creates $|\Phi^+\rangle$. This exact pattern reappears in teleportation.

How it works

- How it works: Since the circuit consists of two qubits both initialized to $|0\rangle$ the initial combined state will be:

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

17

Diagram: How first Bell state starts from $|00\rangle$ (_003 slide 17).

How to read it: Read the column vector as the computational basis representation before gate expansion.

How it Works?

- The Hadamard Gate

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

The Identity Gate:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The Product:

$$H \otimes I = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

19

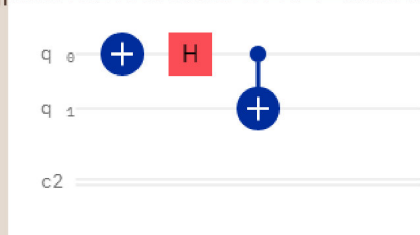
Diagram: Expanded Hadamard calculation (_003 slide 19).

How to read it: For two qubits, the one-qubit H must be expanded using the tensor/Kronecker product with identity before multiplying the two-qubit state.

The Second Bell state

$$|\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$$

- Which is the same as the first Bell state but with a negative phase!. As such this can be implemented easily by using the same circuit as the first Bell state but with the addition of initializing the first qubit to $|1\rangle$ with a Pauli-X gate::



21

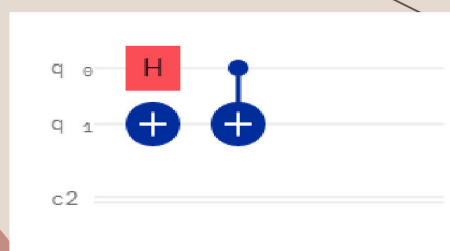
Diagram: Second Bell state circuit (_003 slide 21).

How to read it: The second Bell state differs by phase; Z-type operations create the sign difference.

Third Bell State

$$|\Psi^+\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}}$$

- Same as above, but apply a Pauli x-gate to the second qubits



22

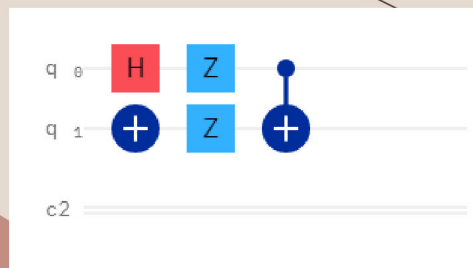
Diagram: Third Bell state circuit (_003 slide 22).

How to read it: Applying an X-style change before/around entanglement switches from the 00/11 pair to the 01/10 pair.

Fourth Bell State

$$|\Psi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}$$

- Add Pauli Z gate to both qubits



23

Diagram: Fourth Bell state circuit (_003 slide 23).

How to read it: The fourth state combines the 01/10 pair with a negative relative phase.

Exam-important

Bell state exam pattern: H on q0 plus CNOT q0 -> q1 gives $(|00\rangle + |11\rangle)/\sqrt{2}$. If a question changes signs or applies X/Z gates, track which basis pair and which relative phase changed.

6. _004 - Quantum Teleportation

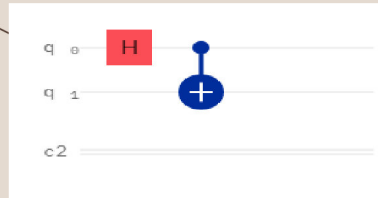
What teleportation means

Quantum teleportation transfers an unknown quantum state from Alice to Bob using a shared entangled pair and two classical bits. It does not move matter and does not allow faster-than-light communication.

Unknown state to transfer:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Teleportation is Strongly Based on Entanglement



5

Diagram: Teleportation is based on entanglement (_004 slide 5).

How to read it: The Bell pair is the resource. Alice and Bob each hold one qubit from the entangled pair before teleportation starts.

Setup and protocol

- 1 Alice has the unknown qubit Q in state $|\psi\rangle$.
- 2 Alice and Bob share an entangled pair A - B , usually $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$. Alice holds A ; Bob holds B .
- 3 Alice applies CNOT from Q to A , then H to Q .
- 4 Alice measures Q and A , producing two classical bits.
- 5 Alice sends those bits to Bob through a classical channel.
- 6 Bob applies correction gates to B , recovering $|\psi\rangle$.

The Setup Process

Teleportation is preceded by a setup process that may be described as follows:

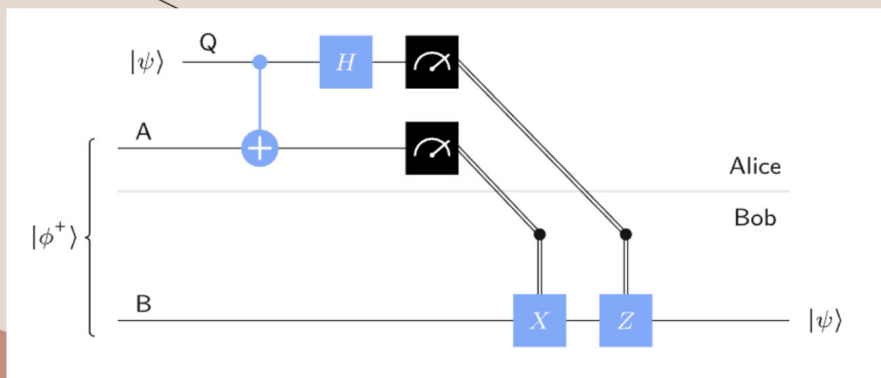
- It is assumed that Alice and Bob share an entangled state.
- Alice holds a qubit **A**, Bob holds a qubit **B**, and together the pair (**A**, **B**) is in the state $|\phi^+\rangle$.
- It could be, for instance, that Alice and Bob were in the same location in the past, they prepared the qubits **A** and **B** in the state $|\phi^+\rangle$, and then each went their own way with their qubit in hand.
- This implies that distance has no effect on the entangled state. Here each of Alice and Bob took his/her qubit away, and perhaps traveled to a different part of the world.
- It may also be the case that the setup of the two qubits has been done in a lab or factory, then distributed to the respective parties.

6

Diagram: Teleportation setup process (_004 slide 6).

How to read it: Keep track of ownership: Alice has Q and A ; Bob has B . The state is transferred to B after Alice's measurement and Bob's correction.

The Teleportation Protocol



7

Diagram: Teleportation protocol circuit (_004 slide 7).

How to read it: The top qubit is the unknown state. The middle and bottom form the Bell pair. Alice's gates and measurements produce bits that control Bob's X/Z corrections.

The Protocol

The teleportation protocol may be described as follows:

1. Alice performs a controlled-NOT operation on the pair (A, Q), with Q being the control and A being the target, and then performs a Hadamard operation on Q.
2. Alice then measures both A and Q, with respect to a standard basis measurement in both cases, and transmits the classical outcomes to Bob. Let us refer to the outcome of the measurement of A as a and the outcome of the measurement of Q as b .
3. Bob receives a and b from Alice, and depending on the values of these bits he performs these operations:
 1. If $a = 1$, then Bob performs a bit flip (or X gate) on his qubit B.
 2. If $b = 1$, then Bob performs a phase flip (or Z gate) on his qubit B.
4. That is, conditioned on ab being 00, 01, 10, or 11, Bob performs one of the operations I, Z, X, or ZX on the qubit B.

9

Diagram: Protocol written as steps (_004 slide 9).

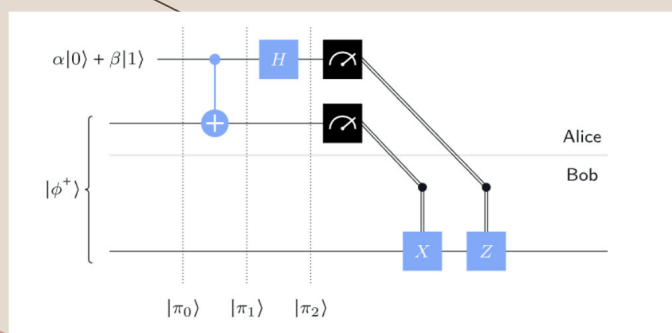
How to read it: This is the algorithmic version: entangle, interact unknown with Alice's half, measure, communicate, correct.

Bob correction table

Alice measurement	Bob correction	Meaning
00	I	Bob already has $ \psi\rangle$.
01	X	Fix a bit flip.
10	Z	Fix a phase flip.
11	X then Z or Z then X	Fix both bit and phase flips. Order differs only by global phase for this purpose.

Applying the protocol : A simple Example

The following diagram shows a simple example that applies the protocol



10

Diagram: Simple teleportation example circuit (_004 slide 10).

How to read it: The diagram applies the abstract protocol to a concrete circuit. The two classical outputs determine the corrections.

Applying the protocol

- Under the assumption that the qubit Q begins the protocol in the state $\alpha|0\rangle + \beta|1\rangle$, the state of the three qubits (B, Q, A) is:

$$|\pi_0\rangle = |\phi^+\rangle \otimes (\alpha|0\rangle + \beta|1\rangle) = \frac{\alpha|000\rangle + \alpha|110\rangle + \beta|001\rangle + \beta|111\rangle}{\sqrt{2}}.$$

- The first gate that is performed is a CNOT gate with control on qubit B and target on qubit Q . This transforms the state $|\pi_0\rangle$ into the expression at π_1 :

$$|\pi_1\rangle = \frac{\alpha|000\rangle + \alpha|110\rangle + \beta|011\rangle + \beta|101\rangle}{\sqrt{2}}.$$

- Then the Hadamard gate is applied, which transforms the state $|\pi_1\rangle$ into (position π_2)

$$|\pi_2\rangle = \frac{\alpha|00\rangle|+\rangle + \alpha|11\rangle|+\rangle + \beta|01\rangle|-\rangle + \beta|10\rangle|-\rangle}{\sqrt{2}}$$

11

Diagram: Teleportation algebra part 1 (_004 slide 11).

How to read it: The algebra expands the combined three-qubit state after Alice's CNOT and H. Terms are grouped by Alice's measurement outcomes.

Applying the Protocol

- Proceeding with above analysis, it will be realized the outcome of the (BAQ) combination is:

$$(\alpha|1\rangle - \beta|0\rangle)|11\rangle.$$

Now Bob Applies ZX on the qubit B to get:

$$(\alpha|0\rangle + \beta|1\rangle)|11\rangle$$

12

Diagram: Teleportation algebra part 2 (_004 slide 12).

How to read it: After measurement, Bob's qubit is one of four related states. X and Z corrections convert it back to $\alpha|0\rangle + \beta|1\rangle$.

Exam-important

Teleportation does not violate the no-cloning theorem. Alice's original state is destroyed by measurement, while Bob's qubit becomes the state after receiving classical bits and applying corrections.

7. _005 - Shor Integer Factorization Algorithm

Problem and importance

Shor's algorithm factors a composite integer N into prime factors. Factoring is important because RSA-style cryptography relies on factoring large numbers being hard for classical computers.

Goal: find p and q such that $N = p \times q$

How it Works

- Choose a random number that is smaller than N
- Then, using quantum computing, find the period of the function $f(x) = a^x \bmod N$

$$a^r \equiv 1 \pmod{N}$$

- The period is the smallest number r such that: $a^r \equiv 1 \pmod{N}$
- Once r is known, it can be used to find the factors of N efficiently.
- Finding the period r is where quantum computing excels.
- A classical computer would need to test many values of x , but a quantum computer can find r quickly using a process called the Quantum Fourier Transform (QFT).
- Once the period is known, the remaining steps are simple math done on a normal computer to find the factors

3

Diagram: High-level period-finding idea (_005 slide 3).

How to read it: Choose a random $a < N$ and study $f(x) = a^x \bmod N$. The period r is the smallest positive r such that $a^r \equiv 1 \bmod N$.

Factoring as period finding

The quantum part finds the period/order r of modular exponentiation efficiently. Classical pre-processing and post-processing surround this quantum subroutine.

Choose a with $1 < a < N$ and $\gcd(a, N) = 1$

$f(x) = a^x \bmod N$

r = smallest positive integer such that $a^r \equiv 1 \pmod{N}$

Overview

Given a large composite number N that is a product of two large primes: :

- Pick a random base (a): Choose a random integer a ($1 < a < N$) that is coprime to N (i.e., $\gcd(a, N) = 1$).
- If not coprime, you've already found a factor via $\gcd$.
- Define the periodic function: Consider the function $f(x) = a^x \bmod N$.
- This function is periodic with some period r , where r is the smallest positive integer such that $a^r \equiv 1 \bmod N$ (this r is called the order of a modulo N).
- Find the period r : This is the key step, done quantumly.
- Once you have r , check if it's even. If not, pick a new a and repeat.

5

Diagram: Shor algorithm overview (_005 slide 5).

How to read it: The algorithm checks easy factors first, then uses period finding. Once r is known, the factors can be recovered using $\gcd$ operations if the conditions are suitable.

Shor Algorithm

- Pick a number a , which is a co-prime with N . (Classical computer);
- Need to find the smallest r such that $a^r \equiv 1 \pmod{N}$.
- If r is even: Go to the next step. If r is odd go back to pick another a .
- If r is even then we found the non-trivial square root of N ;
- Define $x = \text{pow}(a, r/2)$. (on a Classical computer);
- If $x+1 \equiv 0 \pmod{N}$; then one of p, q is contained in $\gcd(x+1, N)$ or $\gcd(x-1, N)$. (Classical computer);
- If these fail (from step 3); we start everything by considering different a .

6

Diagram: Classical post-processing checks (_005 slide 6).

How to read it: If r is odd or $a^{(r/2)}$ is equivalent to $-1 \bmod N$, this attempt may fail and another a is chosen. Failure for one a does not mean Shor failed overall.

The Role of Quantum Computing

- The main point of Shor's algorithm is to find the non-trivial square root of the number that we would like to factorize i.e. N .
- As N gets larger, the calculations become exponentially heavy.
- Shor's great idea was to propose a quantum algorithm to find the order (period) r for which we get the non-trivial square root and then *the whole problem of factoring boils down to order finding*.
- The algorithm partially runs on a quantum computer to find the order r , sandwiched between classical pre and post-processing.
- The next question should be: How we can use quantum computing for order finding?

7

Diagram: Role of quantum computing (_005 slide 7).

How to read it: The hard part is period finding. QFT helps extract a period from the modular exponentiation structure much faster than classical brute force for large N .

Example: factoring 15

$N = 15$, choose $a = 2$

$$2^0 \bmod 15 = 1$$

$$2^1 \bmod 15 = 2$$

$$2^2 \bmod 15 = 4$$

$$2^3 \bmod 15 = 8$$

$$2^4 \bmod 15 = 1 \rightarrow r = 4$$

$$a^{(r/2)} = 2^2 = 4$$

$$\gcd(4 - 1, 15) = \gcd(3, 15) = 3$$

$$\gcd(4 + 1, 15) = \gcd(5, 15) = 5$$

Simple Example for finding the period (order) r

- **Step 1:** $N = 15$.
- **Step 2:** Pick the first coprime $a = 2$ (randomly; $\gcd(2, 15) = 1$, so it's coprime).
- **Step 3:** Compute $f(x) = 2^x \bmod 15$ for $x = 0, 1, 2, \dots$:
 - $2^0 = 1 \bmod 15$
 - $2^1 = 2 \bmod 15$
 - $2^2 = 4 \bmod 15$
 - $2^3 = 8 \bmod 15$
 - $2^4 = 16 \equiv 1 \bmod 15$ (back to 1, so period $r = 4$).
- **Step 4:** $r = 4$ (found via period search; quantumly, this is fast).
- **Step 5:** r is even, so compute $a^{r/2} = 2^2 = 4$.
 - $\gcd(4 - 1, 15) = \gcd(3, 15) = 3$
 - $\gcd(4 + 1, 15) = \gcd(5, 15) = 5$
 - Result: Factors are 3 and 5.

8

Diagram: Worked example for $N=15$ (_005 slide 8).

How to read it: Track the repeated modular values until the sequence returns to 1. The return point gives r , then $\gcd$ gives the factors.

QFT in Shor

QFT turns periodic structure into measurement peaks. In Shor, it makes outcomes near multiples related to Q/r likely. Continued fractions can then recover r from a measured value.

How the Quantum Fourier Transform Speeds the Period Computation

- The speedup in Shor's algorithm comes from using the **Quantum Fourier Transform (QFT)**, a quantum analog of the classical Discrete Fourier Transform (DFT).
- The QFT efficiently extracts periodicities (frequencies) from a superposition of states, allowing period finding in polynomial time instead of exponential.

Why Fourier Transform Helps

- The period r manifests as a repeating pattern in the function $f(x) = a^x \bmod N$.
- In quantum computing, we create a superposition over many x values: $|\psi\rangle = (1/\sqrt{Q}) \sum |x\rangle |a^x \bmod N\rangle$, where Q is a power of 2 larger than N^2 (to ensure accuracy).
- Measuring the second register ($|a^x \bmod N\rangle$) collapses it to some value s , leaving the first register in a superposition of x where $a^x \equiv s \bmod N$. These x are arithmetic progressions with difference r : $x_0, x_0+r, x_0+2r, \dots$, creating a periodic signal with period r .
- Applying QFT to this periodic superposition transforms it into peaks at frequencies that are multiples of Q/r (approximately).
- Measuring after QFT gives a value y close to $k^*(Q/r)$ for integer k , from which we can estimate $1/r$ (hence r) using continued fractions approximation.

9

Diagram: How QFT speeds period computation (_005 slide 9).

How to read it: The periodic function is encoded in amplitudes. QFT converts periodicity in the computational basis into frequency information.

How QFT Speeds the Period Computation

- The QFT is like DFT: For a periodic signal with period r , DFT outputs spikes at frequency $1/r$ (and harmonics). Classically, DFT on M points takes $O(M \log M)$ time via FFT, but to resolve r up to N , M needs to be $\sim N^2$, and evaluating $a^x \bmod N$ for all points is $O(N^2 \log N)$ time—still too slow for large N .
- Quantumly:
- Superposition evaluates $a^x \bmod N$ for all x "in parallel" via quantum parallelism.
- QFT is implemented with $O((\log Q)^2) = O((\log N)^2)$ gates, exponentially faster.
- Overall: Shor's runs in $O((\log N)^3)$ time on a quantum computer.

Simple Illustration with Fourier Transform

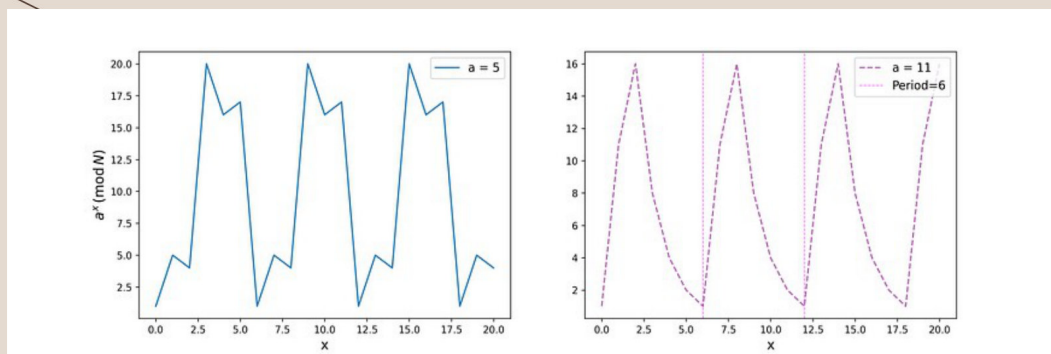
- Suppose a signal like $[1, 0, 0, 1, 0, 0, 1, 0, 0, \dots]$ (period 3 over 9 points).
- DFT/FFT outputs peaks at frequencies 0, $3/9=1/3$, $6/9=2/3$ (multiples of $1/\text{period}$).
- Peak at $1/3$ implies $\text{period}=3$.
- In Shor's, QFT finds these frequencies, and post-processing (continued fractions) extracts r exactly with high probability.
- Thus, QFT turns the hard period search into efficient frequency estimation, enabling quantum factorization to break RSA-sized numbers in seconds on a large quantum computer, vs. years classically.

10

Diagram: QFT period computation process (_005 slide 10).

How to read it: The slide connects superposition, modular exponentiation, QFT, measurement, and classical fraction recovery.

Sample QFT Superpositions



11

Diagram: Sample QFT superposition plots (_005 slide 11).

How to read it: The peaks represent likely measurement outcomes after QFT. Peaks are the signal from which the period is inferred.

Exam-important

Shor is hybrid: classical choice/checks + quantum period finding + classical gcd post-processing. The exponential speedup is tied to efficient period finding through QFT, not to a generic "try all factors" process.

8. _006 - Quantum Fourier Transform (QFT)

DFT vs QFT

The classical Discrete Fourier Transform converts a sequence into frequency components. The Quantum Fourier Transform applies the analogous transform to the amplitudes of a quantum state.

The Discrete Fourier Transform

The Discrete Fourier Transform:

Transforms a sequence of complex numbers

$$\{\mathbf{x}_j\} := x_0, x_1, \dots, x_{N-1}$$

Into another sequence of N:

$$\{\mathbf{X}_k\} := X_0, X_1, \dots, X_{N-1}$$

Using the following relation:

$$X_k = \sum_{j=0}^{N-1} x_j e^{-\frac{2i\pi}{N}kj}$$

3

Diagram: Discrete Fourier Transform formula (_006 slide 3).

How to read it: This is the classical reference point: DFT maps a list of complex numbers into frequency-domain coefficients.

QFT definition

For $N = 2^n$:

$$QFT |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \exp(2\pi i j k / N) |k\rangle$$

The QFT Applied to qubit states

- The definition of the Quantum Fourier Transform is quite analogous to the DFT definition.
- In fact, we consider the computational basis $\{|0\rangle, |1\rangle, \dots, |N-1\rangle\}$ (in decimal notation), with $N=2^n$, where n is the number of qubits.
- The action of the QFT on one of the basis states $|k\rangle$ is described by the following expression::

$$|k\rangle \longrightarrow QFT |k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{\frac{2i\pi}{N}kj} |j\rangle$$

4

Diagram: QFT applied to basis states (_006 slide 4).

How to read it: Each basis state $|j\rangle$ becomes a superposition over all $|k\rangle$ with phase depending on j and k .

Applying to general states and amplitudes

- since the QFT is a linear operator, it acts on a general quantum state $|\psi\rangle$ as follows:

$$|\psi\rangle = \sum_{j=0}^{N-1} x_j |j\rangle \xrightarrow{\mathcal{QFT}} |\tilde{\psi}\rangle = \sum_{k=0}^{N-1} y_k |k\rangle$$

- Each Amplitude is represented by:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \cdot \omega^{kj}, \text{ with } \omega = e^{\frac{2i\pi}{N}}$$

5

Diagram: QFT on a general state (_006 slide 5).

How to read it: For general amplitudes x_j , QFT produces new amplitudes y_k . The transform is linear over the whole superposition.

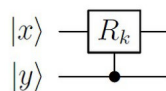
QFT circuit and complexity

The QFT circuit uses Hadamard gates, controlled rotations R_k , and final SWAPs to reverse qubit order.

The QFT Circuit: Combination of Hadamard, SWAP and R^k

Indeed, the QFT can be defined as a combination and succession of three different gates: the Hadamard gate, the SWAP gate, and the *controlled- R_k* (defined below).

$$cR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{\frac{2i\pi}{2^k}} \end{pmatrix}$$

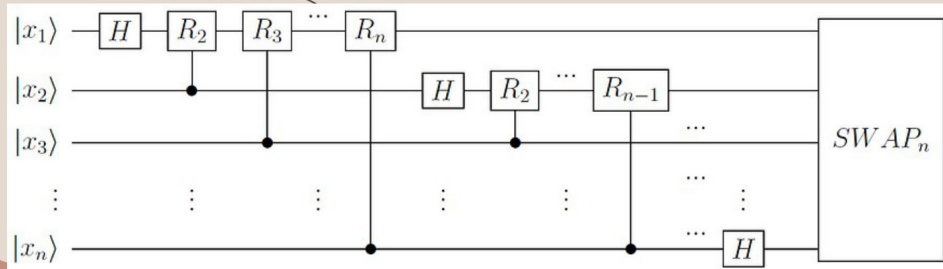


6

Diagram: QFT components: H, SWAP, controlled- R_k (_006 slide 6).

How to read it: Hadamard creates local superposition; controlled rotations add phase relationships; SWAPs fix the output bit order.

The Circuit:



7

Diagram: Full QFT circuit structure (_006 slide 7).

How to read it: Read left to right: each qubit gets H and controlled phase rotations from other qubits. The output order is reversed unless final SWAPs are used.

The Complexity

Complexity is:

Number of operations

$$\frac{n(n+1)}{2} + \frac{n}{2}$$

Implies Complexity is :

$O(n^2)$ in the number of operations, which is a complexity of **$O(\log(N)^2)$** , with N the number of basis states. This quadratic complexity of the QFT is one of the key reasons why Shor's algorithm is polynomial.

8

Diagram: QFT complexity (_006 slide 8).

How to read it: For n qubits, QFT uses $O(n^2)$ gates. Since $N=2^n$, this is polynomial in $\log N$, which is critical for Shor.

Examples

Applying QFT to single qubits:

- Let's apply the quantum Fourier transform to the state .

$$\mathcal{F}|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

- As you can see, becomes a superposition of states with no phase shift.

$$\mathcal{F}|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{\pi i}|1\rangle) = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

9

Diagram: QFT on one qubit (_006 slide 9).

How to read it: For one qubit, QFT acts like a Hadamard transform: $|0\rangle$ goes to $|+\rangle$, and $|1\rangle$ goes to $|-\rangle$.

Applying QFT to two qubits

Let's apply the quantum Fourier transform to quantum state $|01\rangle$ below .

- As you can see, the Quantum Fourier Transform of becomes a superposition where each component has a phase shift of π , indicating a phase-inverted state.

$$\begin{aligned}\mathcal{F}|01\rangle &= \frac{1}{2} \sum_{k=0}^3 e^{2\pi i \cdot 1 \cdot k/4} |k\rangle = \frac{1}{2} (|00\rangle + e^{\frac{\pi i}{2}} |01\rangle + e^{\frac{2\pi i}{2}} |10\rangle + e^{\frac{3\pi i}{2}} |11\rangle) \\ &= \frac{1}{2} (|00\rangle + i|01\rangle - |10\rangle - i|11\rangle)\end{aligned}$$

- Let's apply the quantum Fourier transform to the state $|10\rangle$.

$$\begin{aligned}\mathcal{F}|10\rangle &= \frac{1}{2} \sum_{k=0}^3 e^{2\pi i \cdot 2 \cdot k/4} |k\rangle = \frac{1}{2} (|00\rangle + e^{\pi i} |01\rangle + e^{2\pi i} |10\rangle + e^{3\pi i} |11\rangle) \\ &= \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle)\end{aligned}$$

10

Diagram: QFT on two qubits (_006 slide 10).

How to read it: The two-qubit example shows how phases distribute across $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$.

Exam-important

QFT is exam-important because it extracts periodicity/frequency information from amplitudes. It is the mathematical engine behind Shor's period-finding speedup.

9. _007 - Grover's Algorithm

Problem and speedup

Grover's algorithm searches an unstructured database of N items for a marked item. Classical unstructured search needs $O(N)$ queries in the worst case; Grover uses $O(\sqrt{N})$ oracle queries. This is a quadratic speedup, not exponential.

$N = 2^n$ possible inputs

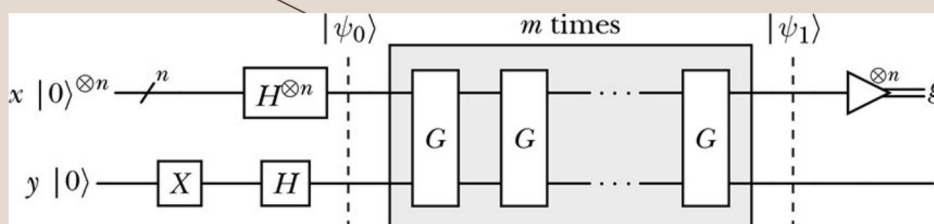
Classical: $O(N)$

Grover: $O(\sqrt{N})$

Optimal iterations for one marked item $\approx (\pi/4) \sqrt{N}$

In a Nutshell: Grover Algorithm

The figure below is a typical circuit for Grover's algorithm



3

Diagram: Typical Grover circuit (_007 slide 3).

How to read it: The circuit starts with superposition, then repeats oracle G and diffusion/inversion steps before measurement.

The main steps

- As the above diagram shows, the process starts with a register x of multiple (n) qubits all in the state $|0\rangle$, and a register y containing a single qubit in the same state.
- Then an X gate is applied to y , and all other n qubits are put into superpositions with H gates.
- Next, we repeatedly apply an oracle operator followed by Diffusion operator

4

Diagram: Main Grover steps (_007 slide 4).

How to read it: Initialize, apply Hadamards, repeatedly apply oracle then diffusion, and measure. One oracle + one diffusion is one Grover iteration.

Main Steps

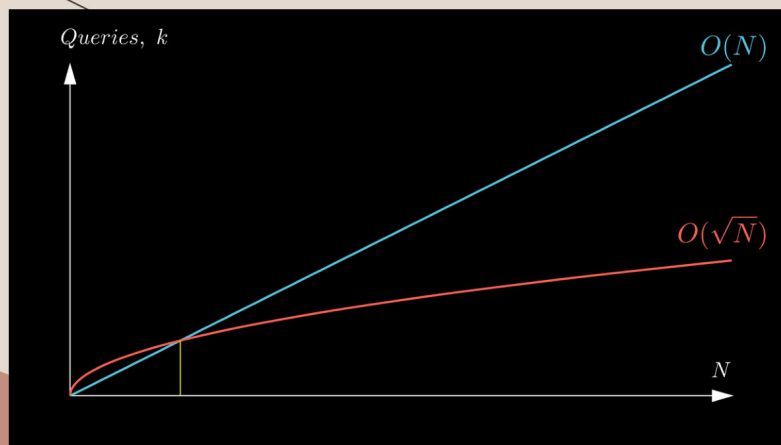
- The main steps may be summarized as follows:
- Start with a register on n qubits all in the $|0\rangle$ state
- Apply a Hadamard gate to each of the n qubits
- Repeat:
 - Apply an Oracle operator
 - Apply a diffusion operator
- Measure all qubits

5

Diagram: Grover algorithm main-step summary (_007 slide 5).

How to read it: Memorize the order: $|0\rangle$ register $\rightarrow$ H superposition $\rightarrow$ oracle $\rightarrow$ diffusion $\rightarrow$ repeat $\rightarrow$ measure.

The Performance of the Algorithm



7

Diagram: Grover performance graph (_007 slide 7).

How to read it: The graph compares $O(N)$ classical search against $O(\sqrt{N})$ Grover search. As N grows, the gap becomes important.

Oracle

The oracle identifies the marked state by flipping its phase. It does not directly output the answer; it marks the answer so interference can amplify it.

Oracle action:

$0|x\rangle = -|x\rangle$, if $x = x_s$

$0|x\rangle = |x\rangle$, if $x \neq x_s$

Oracle Operations

Mathematical Representation of the Oracle

- Let $|x\rangle$ be any quantum state in the computational basis, and let $|x_s\rangle$ denote the marked (solution) state. The Oracle O operates as follows:

$$O|x\rangle = \begin{cases} -|x\rangle & \text{if } x = x_s \\ |x\rangle & \text{otherwise} \end{cases}$$

In Quantum Terms:

- Marked State:** For the state $|x_s\rangle$, the oracle applies a -1 phase shift, flipping its amplitude from $+A$ to $-A$.
- Unmarked States:** For all other states $|x\rangle$, the amplitude remains unchanged.

9

Diagram: Oracle mathematical representation (_007 slide 9).

How to read it: The oracle is a phase flip on the marked state. The state remains normalized; only the sign/phase of the target amplitude changes.

How it Works:

- The oracle's action depends on encoding the condition that identifies the marked state. Let's explore how this works in practice.

1. Encoding the Problem Condition

- To implement the oracle, a quantum circuit is designed to:
- Evaluate a **Boolean function** $f(x)$, where:

$$f(x) = \begin{cases} 1 & \text{if } x = x_s \\ 0 & \text{otherwise} \end{cases}$$

If $f(x)=1$, the oracle flips the phase of $|x\rangle$.

2. Phase Flip Operation

- The oracle achieves the phase flip by using a **controlled operation** that applies a Z-gate (phase shift of π) to the marked state. Mathematically:

$$O = U_f \cdot (I \otimes Z) \cdot U_f^\dagger$$

10

Diagram: How the oracle works (_007 slide 10).

How to read it: The slide separates evaluating $f(x)$ from phase flip. In real circuits, auxiliary qubits may compute $f(x)$ and then be uncomputed to clean up.

Diffusion and amplitude amplification

The diffusion operator reflects amplitudes around the average. After the oracle makes the target amplitude negative, reflection around the average increases the target amplitude.

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle$$

$$D = 2|s\rangle\langle s| - I$$

Amplitude Amplification

- In Grover's algorithm, the objective is to **find a specific marked state** $|x_s\rangle$ among N possible states.
- The challenge lies in the fact that a classical search would take $O(N)$ steps, but Grover's algorithm uses quantum mechanics to solve the problem in $\sqrt{O(N)}$ time.
- Amplitude amplification is the core process that amplifies the amplitude of the marked state and suppresses others, thus making the marked state more likely to be measured. The process consists of two key operations:
- **Oracle Application:** Marks the correct solution by flipping the sign (phase flip) of the marked state's amplitude.
- **Grover Diffusion Operator:** Reflects the quantum state around the average amplitude of all states, enhancing the amplitude of the marked state.
- Each iteration of applying the oracle and diffusion operator boosts the probability of finding the marked state by increasing its amplitude relative to the other states.

13

Diagram: Amplitude amplification (_007 slide 13).

How to read it: Each iteration rotates probability mass toward the marked state. Too many iterations can rotate past the optimum and reduce success probability.

The Diffusion Operator

- The Grover Diffusion Operator is the transformation responsible for amplifying the amplitude of the marked state. It is also called the **reflection operator** because it reflects the state across the average amplitude of all possible states.
- **Mathematical Definition:**
- The Grover Diffusion Operator is mathematically expressed as:

$$D = 2|\psi_0\rangle\langle\psi_0| - I$$

Where:

- $|\psi_0\rangle$ is the initial quantum state after applying the Hadamard transform to all qubits. It represents the **uniform superposition** of all possible states,
- I is the **identity operator**, which leaves all states unchanged.
- $|\psi_0\rangle\langle\psi_0|$ is the **projection operator** onto the state $|\psi_0\rangle$, effectively projecting the quantum state onto the subspace of the uniform superposition.

14

Diagram: Diffusion operator (_007 slide 14).

How to read it: Read $D = 2|s\rangle\langle s| - I$ as reflection about the uniform superposition state. Operationally, it performs inversion about the mean amplitude.

Step-by-Step Reflection

- **Before the Diffusion Operator:** Initially, the quantum state $|\psi\rangle$ is a superposition of all states, and the amplitudes are the same for each state (if the oracle has not been applied yet). Let's denote the state after the oracle application as:

$$|\psi\rangle = \alpha|x_s\rangle + \sum_{x \neq x_s} \beta|x\rangle$$

- Where:
 - α is the amplitude of the marked state $|x_s\rangle$,
 - β is the amplitude of the unmarked states.
- **Average Amplitude:** The average amplitude $\langle\psi_0\rangle$ of all states in the superposition is the mean of the amplitudes. Initially, when no oracle is applied, all states have the same amplitude, and the average amplitude is:

$$\langle\psi_0\rangle = \frac{1}{\sqrt{N}}$$

- After the oracle is applied, the average amplitude changes, but it still reflects the uniformity of the quantum state before amplification.

15

Diagram: Step-by-step reflection (_007 slide 15).

How to read it: Before diffusion, the marked amplitude is phase-flipped. The reflection then boosts it relative to unmarked amplitudes.

Reflection

- **Reflection:** The operator D reflects the quantum state about this average. This reflection operates on all states $|x\rangle$, including the marked and unmarked states. Mathematically, it can be viewed as the following:

$$D|\psi\rangle = 2|\psi_0\rangle\langle\psi_0||\psi\rangle - I|\psi\rangle$$

- The first part, $|\psi_0\rangle\langle\psi_0|$, projects the state onto $|\psi_0\rangle$ and then multiplies by 2, effectively flipping the amplitudes of all states concerning the average.
- The second part, $-I|\psi\rangle$, subtracts the identity operator from the previous term, ensuring that the state is reflected in the average amplitude.
- **Effect on the Marked State:**
 - For the marked state $|x_s\rangle$, the reflection amplifies its amplitude. Since it was flipped in phase by the oracle, this reflection boosts the amplitude of the marked state while reducing the amplitudes of the unmarked states.
 - For unmarked states, the reflection operator suppresses their amplitudes.

16

Diagram: Reflection explanation (_007 slide 16).

How to read it: The target amplitude becomes larger after reflection while unmarked amplitudes are suppressed. This is constructive/destructive interference in action.

Exam-important

Grover's key formula: about $(\pi/4)\sqrt{N}$ iterations for one marked item. Oracle alone does not solve the problem; oracle plus diffusion creates amplitude amplification.

10. _008 - Deutsch-Jozsa Algorithm Revisited

Promise problem

Deutsch-Jozsa determines whether a function is constant or balanced, assuming one of those two cases is guaranteed.

- Constant: $f(x)$ is the same for every input x .
- Balanced: $f(x)=0$ for exactly half the inputs and $f(x)=1$ for the other half.
- Classical deterministic worst case: $2^{(n-1)}+1$ queries.
- Quantum: one oracle query.

The Problem

Table 9-1: Four possible oracles

x	$f_0(x)$	$f_1(x)$	$f_2(x)$	$f_3(x)$
000	0	1	0	1
001	0	1	0	1
010	0	1	0	0
011	0	1	0	1
100	0	1	1	0
101	0	1	1	0
110	0	1	1	0
111	0	1	1	1

3

Diagram: Deutsch-Jozsa truth table example (_008 slide 3).

How to read it: The table shows possible outputs. The problem is not to learn every row; it is to decide if the whole function is constant or balanced.

Circuit idea

The algorithm uses a superposition over all inputs, an output/helper qubit prepared as $|-\rangle$, an oracle, and interference from a second Hadamard layer.

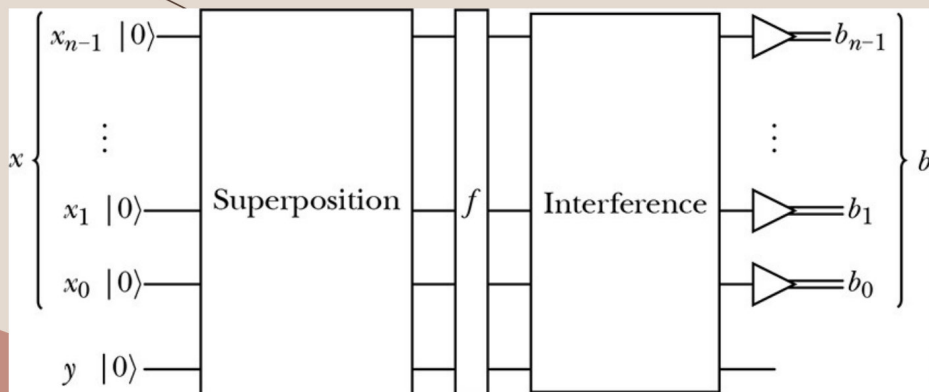
Input register: $|0\rangle^{\otimes n}$

Helper qubit: $|1\rangle$

After H gates: $(1/\sqrt{2^n}) \sum_x |x\rangle \otimes |-\rangle$

Oracle phase kickback: $|x\rangle|-\rangle \rightarrow (-1)^{f(x)} |x\rangle|-\rangle$

Initial High Level Design of the Circuit

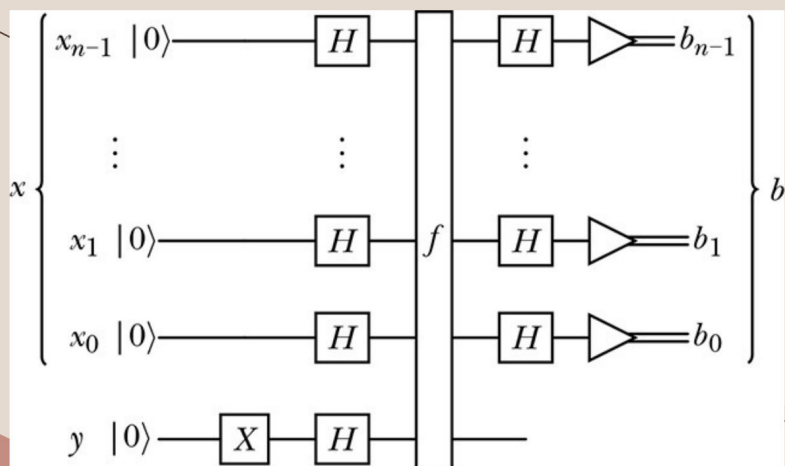


5

Diagram: Initial high-level circuit (_008 slide 5).

How to read it: The superposition block prepares all inputs. The interference block uses phase differences to reveal whether outputs were constant or balanced.

The Detailed Algorithm Circuit



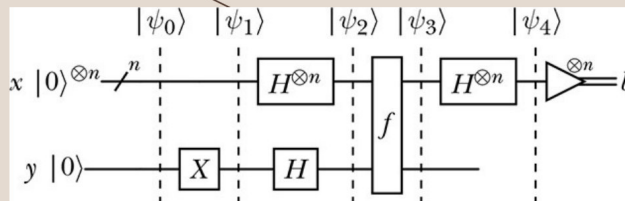
6

Diagram: Detailed algorithm circuit (_008 slide 6).

How to read it: Each input qubit receives H before and after the oracle. The helper qubit starts at $|1\rangle$, gets X/H -style preparation into $|-\rangle$, and supports phase kickback.

A More Compact Representation of the Circuit

- The figure below is more compact representation of the algorithm



In this figure all x qubits are collectively drawn on a single row, applying the exponent tensor product n to each gate to stand for n gates.

7

Diagram: Compact circuit representation (_008 slide 7).

How to read it: Multiple qubits can be drawn as a bundle to simplify diagrams. The bundle represents tensor products of n qubits, not a single large wire.

Comments on the above Diagram

There are two new conventions in the above diagram (previous slide):

- First, to represent a *bundle* of qubits we use a single wire but place a slash on it, along with the number of wires (or qubits) in the bundle.
- Here, the upper-left horizontal line represents a bundle of n qubits. This declaration continues for that wire through the rest of the circuit, unless it's explicitly changed.
- Therefore, in the above diagram, the upper line coming out of f still represents a bundle of n qubits. That bundle then goes into a bundle of n meters. The collective output of the meters is a bitstring of n classical bits.
- Treating multiple qubits as a group in this way often makes the diagrams simpler and easier to interpret. We call each such bundle of qubits a **register**.

Diagram: Diagram conventions comments (_008 slide 8).

How to read it: This slide explains how to read bundled wires and the n -exponent notation. Do not confuse the bundle with one physical qubit.

Comments on the Above Diagram

- The second new notation is the $\otimes n$ exponent. This indicates that a state, gate, or meter is tensored with itself n times.
- For example, we can write $|0\rangle^{\otimes 3} = |0\rangle \otimes |0\rangle \otimes |0\rangle = |000\rangle$, and $H^{\otimes 3} = H \otimes H \otimes H$. Similarly, a meter with $\otimes 3$ above and to its right represents a stack of three meters, applied to each of three qubits.
- In this case, the input $|0\rangle^{\otimes n}$ tells us that x is a system created by multiplying together n qubits, each in state $|0\rangle$. This bundle is then run through $H^{\otimes n}$, a vertical system of gates composed of n copies of H stacked up and multiplied together.
- They all go through f , along with y . The qubits of x are then fed into another system of n copies of H , and then each qubit is measured independently, creating the n -bit output bitstring b .

9

Diagram: More diagram notation comments (_008 slide 9).

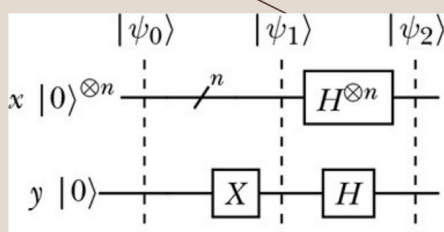
How to read it: $H^{\otimes n}$ means apply H to each input qubit. The meter symbol with n bits means each input qubit is measured separately.

Step flow

- 1 Prepare n input qubits as $|0\rangle$ and one helper qubit as $|1\rangle$.
- 2 Apply H to all qubits. The helper becomes $|-\rangle$.
- 3 Query the oracle once. Because the helper is $|-\rangle$, the function value becomes phase $(-1)^{f(x)}$.
- 4 Apply H to the input register again.
- 5 Measure the input register. All zeros means constant; any nonzero result means balanced.

Step 1 Part 1: Before the Oracle

- All qubits start out as $|0\rangle$.
- The y qubit is inverted by an X gate, and then each qubit in both x and y goes into an H gate, as shown in the figure below:



$$\begin{aligned}\psi_0 &= |0\rangle^{\otimes n}|0\rangle \\ \psi_1 &= |0\rangle^{\otimes n}|1\rangle \\ \psi_2 &= |+\rangle^{\otimes n}|-\rangle\end{aligned}$$

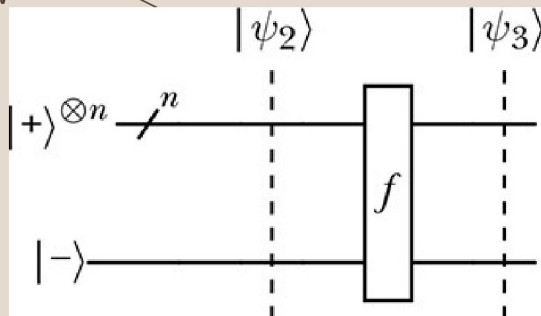
11

Diagram: Before the oracle (_008 slide 11).

How to read it: The state is a superposition over all inputs with the helper arranged for phase kickback.

Step 2: Querying the Oracle

- Now that we have placed x *have been placed* into an equal superposition of all possible inputs for n qubits, and y have been placed into $|-\rangle$, we're ready to give those inputs to the oracle. The diagram for this operation is shown below



13

Diagram: Querying the oracle (_008 slide 13).

How to read it: The oracle encodes function values into phase. This is why one query can reveal a global property of the function.

The Oracle Role

- As hinted before, and based on quantum superposition parallelism, the oracle will process each of the states in the superposition $|\psi_2\rangle$ at the same time and produce a new superposition $|\psi_3\rangle$ as output.
- Each state in the output superposition is the result of querying the oracle with the corresponding state in the input superposition.
- The result of applying The oracle, *in one step*, acts in parallel on every possible input of n qubits to produce a new superposition that contains the output of every input qubit.
- Each such output has the same form: the input state $|k\rangle$ multiplied with the y input $|-\rangle$, multiplied by either 1 or -1, depending on the oracle's response to that input.

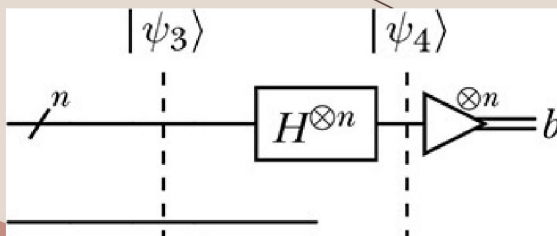
14

Diagram: Oracle role explanation (_008 slide 14).

How to read it: If $f(x)=0$, the phase is +1; if $f(x)=1$, the phase is -1. Constant functions preserve uniform alignment; balanced functions create cancellation at the all-zero outcome.

Step 3: Preprocessing & Measurement

- The postprocessing after measurement is very complex for this problem in it is out of context to go through it in this course
- The part of the circuit that will carry out the measurement process is given below:



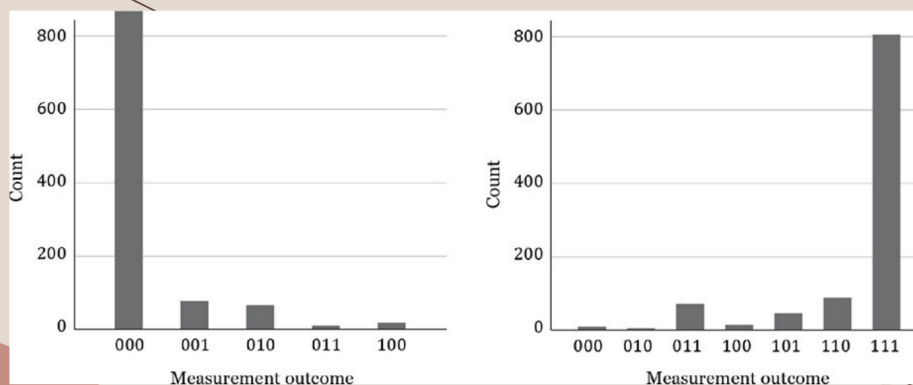
15

Diagram: Post-processing and measurement (_008 slide 15).

How to read it: The second H layer converts phase structure into measurement probabilities. Measurement of all zeros indicates constant.

The Results

- The results of the process are shown in the histograms below



16

Diagram: Deutsch-Jozsa result histograms (_008 slide 16).

How to read it: The histogram concentrated at 000 corresponds to constant; distributions away from 000 indicate balanced behavior.

Exam-important

Deutsch-Jozsa is high-yield because it demonstrates phase kickback and interference. Rule to memorize: after the final H, measuring 00...0 means constant; anything else means balanced.

11. _009 - Quantum Ripple Adder

What a ripple adder does

A ripple-carry adder adds binary numbers bit by bit. Each bit produces a sum bit and a carry bit. The carry from bit i becomes the carry-in to bit $i+1$, so the carry “ripples” through the number.

Sum bit: $S_i = A_i \oplus B_i \oplus C_i$

Carry: $C_{i+1} = (A_i \text{ AND } B_i) \text{ OR } (C_i \text{ AND } (A_i \oplus B_i))$

Gate Definitions and Roles

Gate Definitions and Roles

Quantum Gates Used:

1.CNOT (Controlled-X) Gate:

1. Flips the target qubit if the control qubit is $|1\rangle$
2. Used for XOR operations and entanglement
3. Essential for implementing classical XOR logic

2.Toffoli (CCX) Gate:

1. Three-qubit gate that flips the target if both control qubits are $|1\rangle$
2. Implements classical AND logic
3. Used to generate carry bits

3.X Gate:

1. Bit flip gate (NOT operation)
2. Used for input preparation and corrections

3

Diagram: Gate definitions and roles (_009 slide 3).

How to read it: CNOT implements XOR-style operations. Toffoli implements AND-style carry conditions. X gates prepare input bits set to 1.

How it Works?

How the Ripple Carry Adder Works

- The ripple carry adder operates by implementing the classical binary addition algorithm:

Bit-by-bit Addition: Starting from the least significant bit (LSB), each position adds:

- Bit from first number (A)
- Bit from second number (B)
- Carry from previous position

Full Adder Logic: For each bit position:

- **Sum bit:** $S = A \oplus B \oplus \text{Carry}_{in}$
- **Carry out:** $\text{Carry}_{out} = (A \wedge B) \vee (\text{Carry}_{in} \wedge (A \oplus B))$
- **Carry Propagation:** The carry-out from position i becomes the carry-in for position $i+1$

4

Diagram: How ripple-carry addition works (_009 slide 4).

How to read it: Each bit position computes sum and carry-out. The carry-out becomes the next carry-in, which is why the circuit depth grows with bit width.

Quantum Implementation Details

Qubit Allocation:

- $a[0 : n-1]$: First input number qubits
- $b[0 : n-1]$: Second input number qubits
- $c[0 : n]$: Carry qubits (including final carry-out)
- $s[0 : n-1]$: Sum result qubits

Gate Operations:

- **CX gates** create the XOR operations needed for sum calculation
- **CCX (Toffoli) gates** generate carry bits using AND logic
- **Barrier instructions** organize the circuit for clarity

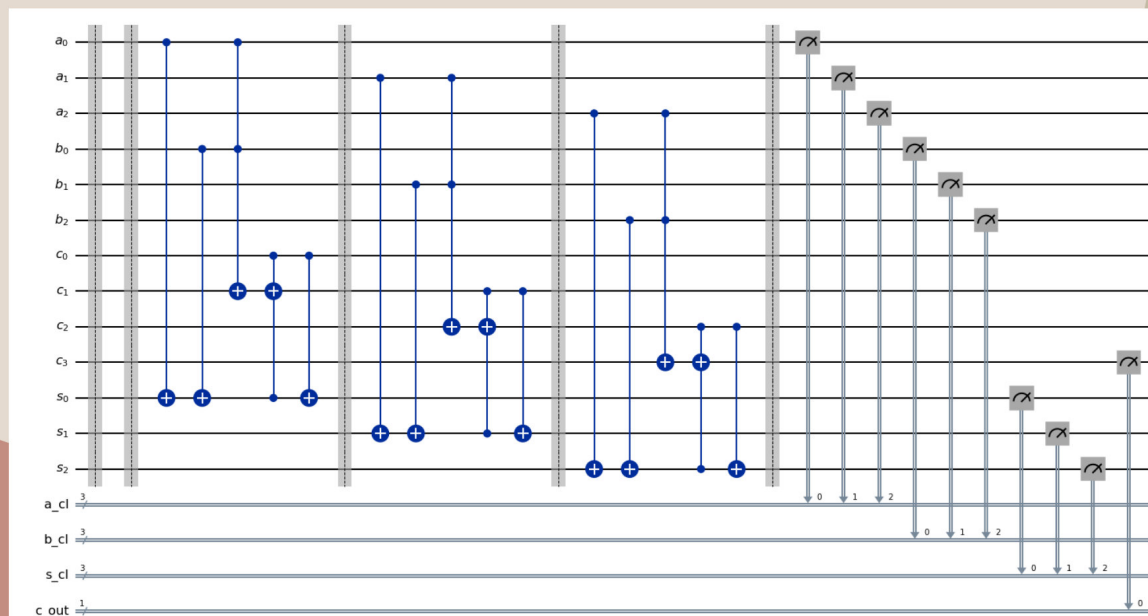
5

Diagram: Quantum implementation details (_009 slide 5).

How to read it: Registers store input A, input B, carry qubits, and sum output qubits. Ancilla/carry qubits are needed to keep the operation reversible.

Circuit and code structure

Circuit Diagram



6

Diagram: Ripple adder circuit diagram (_009 slide 6).

How to read it: The repeated pattern shows full-adder blocks across bit positions. Blue dots represent controls; target symbols show where values are flipped.

Create Adder Circuit

Create the quantum ripple carry adder circuit

Qubit allocation:

- a[i]: First input number (bits 0 to n-1)
- b[i]: Second input number (bits 0 to n-1)
- c[i]: Carry bits (bits 0 to n, where c[0] is carry-in, c[n] is final carry-out)
- s[i]: Sum output bits (bits 0 to n-1)

8

Diagram: create_adder_circuit structure (_009 slide 8).

How to read it: The code allocates registers: a, b, c, and s. It then composes full-adder stages and final carry handling.

Create Circuit

```
def create_adder_circuit(self):
    # Create quantum registers
    a = QuantumRegister(self.n_bits, 'a') # First input
    b = QuantumRegister(self.n_bits, 'b') # Second input
    c = QuantumRegister(self.n_bits + 1, 'c') # Carry bits (including carry-out)
    s = QuantumRegister(self.n_bits, 's') # Sum output
    # Classical registers for measurement
    a_cl = ClassicalRegister(self.n_bits, 'a_cl')
    b_cl = ClassicalRegister(self.n_bits, 'b_cl')
    s_cl = ClassicalRegister(self.n_bits, 's_cl')
    c_out_cl = ClassicalRegister(1, 'c_out')
    # Measure all qubits
    self.circuit.measure(a, a_cl)
    self.circuit.measure(b, b_cl)
    self.circuit.measure(s, s_cl)
    self.circuit.measure(c[self.n_bits], c_out_cl) # Final carry-out
```

9

Diagram: Create circuit code details (_009 slide 9).

How to read it: The circuit creates quantum and classical registers and prepares carry/sum measurement points. Barriers separate preparation, computation, and measurement for readability.

Create Circuit

```
# Create the circuit
self.circuit = QuantumCircuit(a, b, c, s, a_cl, b_cl, s_cl, c_out_cl)
# Add a barrier for clarity
self.circuit.barrier()
self.circuit.label = "Input Preparation"
# Implement the ripple carry adder logic
for i in range(self.n_bits):
    self.add_full_adder(i, a, b, c, s)
# Add final barrier
self.circuit.barrier()
```

10

Diagram: Looping through full adders (_009 slide 10).

How to read it: The for-loop applies the full-adder logic for each bit position and propagates the carry from one stage to the next.

Full adder logic

A quantum full adder must compute sum and carry reversibly. CNOT gates build XOR values; Toffoli gates create carry terms.

Full Adder

Add a full adder for the given bit position

Full Adder Logic:

$$\text{sum} = a \oplus b \oplus \text{carry_in}$$

$$\text{carry_out} = (a \wedge b) \vee (\text{carry_in} \wedge (a \oplus b))$$

Quantum Implementation:

1. Compute $a \oplus b$ and store in sum qubit
2. Compute carry_out using Toffoli gates
3. Apply final XOR with carry_in to get final

11

Diagram: Full adder pseudocode (_009 slide 11).

How to read it: Compute $a \oplus b$ into the sum qubit, compute carry-out with Toffoli operations, then include carry-in with CNOT/XOR logic.

Full Addder

```
def add_full_adder(self, bit_position, a, b, c, s):
    i = bit_position
    # Label this section
    self.circuit.barrier()
    # Step 1: Compute  $a[i] \oplus b[i] \rightarrow s[i]$ 
    self.circuit.cx(a[i], s[i])
    self.circuit.cx(b[i], s[i])
    # Step 2: Compute  $\text{carry\_out} = (a[i] \wedge b[i]) \vee (c[i] \wedge (a[i] \oplus b[i]))$ 
    # First part:  $a[i] \wedge b[i] \rightarrow c[i+1]$ 
    self.circuit.ccx(a[i], b[i], c[i+1])
    # Second part:  $c[i] \wedge (a[i] \oplus b[i]) \rightarrow c[i+1]$ 
    # We use s[i] which currently holds  $a[i] \oplus b[i]$ 
    self.circuit.ccx(c[i], s[i], c[i+1])
    # Step 3: Final sum =  $(a[i] \oplus b[i]) \oplus c[i] = s[i] \oplus c[i]$ 
    self.circuit.cx(c[i], s[i])
```

12

Diagram: Full addder code implementation (_009 slide 12).

How to read it: The code uses ccx for Toffoli and cx for CNOT. This maps directly onto AND and XOR logic needed for binary addition.

Gate mapping:

CNOT(a, s) and CNOT(b, s) -> XOR contribution to sum

Toffoli(a, b, c_out) -> carry from A AND B

Toffoli/carry logic -> carry from C_in AND (A XOR B)

Inputs, simulation, and results

Define Inputs

```
def set_input_values(self, a_val, b_val, carry_in=0):
    # Reset circuit to clean state
    self.create_adder_circuit()
    # Convert to binary and set input qubits
    a_binary = format(a_val, f'0{self.n_bits}b')[::-1] # Reverse for LSB first
    b_binary = format(b_val, f'0{self.n_bits}b')[::-1]
    # Create a new circuit with input preparation
    input_prep = QuantumCircuit(self.circuit.qregs[0], self.circuit.qregs[1],
                                self.circuit.qregs[2], self.circuit.qregs[3])
```

14

Diagram: Input preparation code (_009 slide 14).

How to read it: X gates set qubits to represent input bits equal to 1. The code converts decimal values into binary strings for register preparation.

Define Inputs

```
# Set input a
for i, bit in enumerate(a_binary):
    if bit == '1':
        input_prep.x(self.circuit.qregs[0][i]) # a register
# Set input b
for i, bit in enumerate(b_binary):
    if bit == '1':
        input_prep.x(self.circuit.qregs[1][i]) # b register
# Set carry-in
if carry_in == 1:
    input_prep.x(self.circuit.qregs[2][0]) # c register, first qubit

# Combine input preparation with main circuit
full_circuit = input_prep.compose(self.circuit)
self.circuit = full_circuit

return self.circuit
```

15

Diagram: Combining prepared inputs with the adder (_009 slide 15).

How to read it: The prepared input circuit is composed with the adder circuit. Measurements then read sum and carry-out.

Simulate

```
def simulate(self, shots=1024):
    simulator = AerSimulator()
    # Use the preset pass manager for optimization
    pm = generate_preset_pass_manager(backend=simulator, optimization_level=1)
    transpiled_circuit = pm.run(self.circuit)
    # Run simulation
    job = simulator.run(transpiled_circuit, shots=shots)
    result = job.result()
    counts = result.get_counts()
    return counts
```

16

Diagram: Simulation code (_009 slide 16).

How to read it: The simulation transpiles the circuit and runs shots on a simulator, then extracts count results. The most frequent bitstring is interpreted as the answer.

Demonstrate

Demonstration and testing

Demonstrate the quantum ripple carry adder with different examples

```
"""
print("=" * 60)
print("QUANTUM RIPPLE CARRY ADDER DEMONSTRATION")
print("=" * 60)
# Create a 3-bit adder
adder = QuantumRippleAdder(n_bits=3)
# Test different addition operations
test_cases = [
    (3, 2), # 011 + 010 = 101 (5)
    (5, 3), # 101 + 011 = 1000 (8, with carry)
    (7, 1), # 111 + 001 = 1000 (8, with carry)
    (0, 0), # 000 + 000 = 000 (0)
]
```

17

Diagram: Demonstration cases (_009 slide 17).

How to read it: The test cases include 3+2, 5+3, 7+1, and 0+0. They check normal carry and final carry-out cases.

Demonstrate ...

```
for a_val, b_val in test_cases:
    print(f"\nTesting: {a_val} + {b_val}")
    print(f"Expected result: {a_val + b_val}")
    # Set up the circuit with input values
    adder.set_input_values(a_val, b_val)
    # Simulate
    counts = adder.simulate(shots=1024)
    # Analyze results
    adder.analyze_results(counts)
    print("-" * 40)
# Draw the circuit for the last test case
print("\nQuantum Circuit Diagram:")
print("(Showing circuit for the last test case: 7 + 1)")
```

18

Diagram: Demonstration output formatting (_009 slide 18).

How to read it: The code reports expected results and draws the circuit for the last test. Bit order/endian interpretation must be handled carefully.

Demonstrate ...

```
try:
    # Try matplotlib output first
    fig = adder.draw_circuit(output='mpl')
    if fig:
        plt.figure(figsize=(15, 8))
        plt.tight_layout()
        print("Circuit diagram saved as matplotlib figure")
except:
    # Fallback to text output
    print("Matplotlib not available, showing text representation:")
    print(adder.draw_circuit(output='text'))

return adder
```

19

Diagram: Circuit drawing fallback code (_009 slide 19).

How to read it: The slide shows plotting and text fallback for circuit visualization. This is presentation code, not the core adder logic.

Some Results

=====

QUANTUM RIPPLE CARRY ADDER DEMONSTRATION

=====

Testing: 3 + 2

Expected result: 5

Simulation Results Analysis:

A=6, B=2 → Sum=5, Carry=0 → Result=5 (Count: 1024)

Testing: 5 + 3

Expected result: 8

Simulation Results Analysis:

A=5, B=6 → Sum=0, Carry=1 → Result=8 (Count: 1024)

20

Diagram: Sample adder results: 3+2 and 5+3 (_009 slide 20).

How to read it: The simulation returns the expected sums. For 5+3=8, final carry-out is required because 8 exceeds three bits.

Some results

Testing: $7 + 1$

Expected result: 8

Simulation Results Analysis:

 $A=7, B=4 \rightarrow \text{Sum}=0, \text{Carry}=1 \rightarrow \text{Result}=8$ (Count: 1024)

Testing: $0 + 0$

Expected result: 0

Simulation Results Analysis:

 $A=0, B=0 \rightarrow \text{Sum}=0, \text{Carry}=0 \rightarrow \text{Result}=0$ (Count: 1024)

21

Diagram: Sample adder results: $7+1$ and $0+0$ (_009 slide 21).

How to read it: $7+1=8$ is another final-carry case; $0+0$ verifies the zero baseline.

Circuit Analysis

=====

CIRCUIT ANALYSIS

=====

Total qubits used: 13

Circuit depth: 16

Number of gates: 30

Gate count breakdown:

barrier: 5

ccx: 6

cx: 9

measure: 10

22

Diagram: Circuit analysis summary (_009 slide 22).

How to read it: For the shown 3-bit example: 13 total qubits, depth 16, 30 gates, 6 CCX, 9 CX, 10 measurements, and 5 barriers.

Exam-important

Ripple-adder exam focus: know the classical equations, the role of CNOT as XOR, the role of Toffoli as AND/carry, and why carry qubits are needed. Do not get trapped by bit-order formatting in simulator output.

12. Key terms

Term	Short definition
Qubit	Quantum bit. Before measurement, a qubit can be a superposition $\alpha 0\rangle + \beta 1\rangle$.
Amplitude	Complex coefficient of a basis state. Probability is squared magnitude of amplitude.
Superposition	Linear combination of basis states.
Measurement	Process that returns a classical outcome according to probabilities and changes the state.
Coherence	Stable phase relationship that allows interference and superposition.
Decoherence	Loss of quantum behavior due to unwanted interaction/noise.
Entanglement	Multi-qubit state not separable into individual qubit states.
Tensor product	Mathematical operation for combining quantum systems.
Unitary gate	Reversible operation that preserves total probability.
Hadamard H	Creates equal superposition and enables interference.
Pauli-X	Bit-flip gate, quantum NOT.
Pauli-Z	Phase-flip gate: $ 1\rangle$ gets a minus sign.
Phase gate	Changes relative phase without changing immediate measurement probabilities.
CNOT / CX	Controlled-X: flips target when control is $ 1\rangle$.
Toffoli / CCX	Two-control X gate; reversible AND/carry logic.
Bell state	Maximally entangled two-qubit state.
No-cloning theorem	Unknown quantum states cannot be copied perfectly.
Quantum teleportation	Transfer of an unknown state using entanglement, measurement, classical bits, and correction gates.
Oracle	Black-box operation encoding a problem. Often marks states by phase.
Phase kickback	Function value appears as a phase on the input register.
QFT	Quantum Fourier Transform; converts periodic structure into frequency/phase information.
Period/order r	Smallest positive r such that $a^r \equiv 1 \pmod N$.
Diffusion operator	Grover operator $D=2 s\rangle\langle s -I$ that reflects amplitudes around the average.
Amplitude amplification	Repeated oracle+diffusion process that increases marked-state probability.
Ripple carry	Carry-out of one bit becomes carry-in to the next bit.

13. Quick revision cheat sheet

Qubit:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1$$

Hadamard:

$$H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}, \quad H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$$

X gate:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

Z gate:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

CNOT:

control=1 -> flip target; control=0 -> leave target

Bell state from $|00\rangle$:

$$|00\rangle \rightarrow H \text{ on } q_0 \rightarrow (|00\rangle + |10\rangle)/\sqrt{2} \rightarrow \text{CNOT} \rightarrow (|00\rangle + |11\rangle)/\sqrt{2}$$

Teleportation corrections:

00 -> I

01 -> X

10 -> Z

11 -> XZ or ZX (global phase difference only)

Shor:

$$f(x) = a^x \pmod N$$

Find r where $a^r \equiv 1 \pmod N$

Factors from $\gcd(a^{r/2}-1, N)$ and $\gcd(a^{r/2}+1, N)$

QFT:
 $|j\rangle \rightarrow (1/\sqrt{N}) \sum_k \exp(2\pi i j k / N) |k\rangle$

Grover:
 $O(\sqrt{N})$ search, iterations $\approx (\pi/4)\sqrt{N}$
 $D = 2|s\rangle\langle s| - I$

Deutsch-Jozsa:
all-zero measurement $\rightarrow$ constant; nonzero $\rightarrow$ balanced

Ripple adder:
 $S_i = A_i \oplus B_i \oplus C_i$
 $C_{i+1} = (A_i \wedge B_i) \vee (C_i \wedge (A_i \oplus B_i))$

14. Possible exam questions with answers

1. What makes a qubit different from a classical bit?

A bit has one definite value, 0 or 1. A qubit can be $\alpha|0\rangle + \beta|1\rangle$, and measurement returns outcomes with probabilities $|\alpha|^2$ and $|\beta|^2$.

2. Why must $|\alpha|^2 + |\beta|^2 = 1$?

Because total probability of all measurement outcomes must be 1.

3. What does the Hadamard gate do?

It maps $|0\rangle$ to $(|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ to $(|0\rangle - |1\rangle)/\sqrt{2}$, creating equal superposition with phase difference.

4. What is the correct CNOT rule?

The target flips if and only if the control qubit is $|1\rangle$.

5. What is an entangled state?

A multi-qubit state that cannot be written as a tensor product of separate one-qubit states.

6. Write the four Bell states.

$\Phi^+ = (|00\rangle + |11\rangle)/\sqrt{2}$, $\Phi^- = (|00\rangle - |11\rangle)/\sqrt{2}$, $\Psi^+ = (|01\rangle + |10\rangle)/\sqrt{2}$, $\Psi^- = (|01\rangle - |10\rangle)/\sqrt{2}$.

7. How is the first Bell state created from $|00\rangle$?

Apply H to the first qubit, then CNOT with the first qubit as control and the second as target.

8. Why does teleportation not allow faster-than-light communication?

Bob cannot recover the state until Alice sends two classical bits; classical communication is speed-limited.

9. What is the no-cloning theorem and how does teleportation respect it?

Unknown quantum states cannot be copied. Teleportation destroys Alice's original through measurement while Bob receives the state after correction.

10. What is the quantum part of Shor's algorithm?

Efficient period finding for $f(x) = a^x \bmod N$, using modular exponentiation and QFT.

11. Factor 15 using $a=2$ in Shor-style logic.

$2^4 \bmod 15 = 1$, so $r=4$. $a^{(r/2)}=4$. $\gcd(3,15)=3$ and $\gcd(5,15)=5$. Factors are 3 and 5.

12. What does QFT do conceptually?

It transforms amplitude patterns into frequency/phase information and is especially useful for detecting periodicity.

13. What speedup does Grover provide?

It reduces unstructured search from $O(N)$ to $O(\sqrt{N})$.

14. What does Grover's oracle do?

It flips the phase of the marked state while leaving unmarked states unchanged.

15. What does the diffusion operator do?

It reflects amplitudes around the average, amplifying the marked state after the oracle phase flip.

16. What problem does Deutsch-Jozsa solve?

It decides whether a promised Boolean function is constant or balanced.

17. What measurement result means Deutsch-Jozsa is constant?

All zeros in the input register after the final Hadamards.

18. Which gates implement a ripple-carry adder?

CNOT gates implement XOR/sum behavior; Toffoli gates implement AND/carry behavior; X gates prepare input bits.

19. Why are phase gates important if measurement probabilities do not immediately change?

Because relative phase controls later interference, which changes final measurement probabilities.

20. What is the role of controlled gates in algorithms?

They condition operations on other qubits and are essential for entanglement, oracles, arithmetic, and reversible logic.

15. Highest-priority focus topics

- 1 Qubit state math: amplitudes, probabilities, normalization, measurement.
- 2 Core gates: H, X, Y, Z, phase, S, T, U, SWAP, CNOT, Toffoli, Fredkin.
- 3 Bloch sphere: θ , ϕ , phase, and why phase matters.
- 4 Tensor products: product state versus entangled state.
- 5 CNOT truth table and its role in entanglement.
- 6 Bell states: formulas and H+CNOT construction.
- 7 Teleportation: setup, measurements, two classical bits, correction table.
- 8 Shor: period finding, QFT role, gcd post-processing, $N=15$ example.
- 9 QFT: definition, circuit components, $O(n^2)$ gate complexity.
- 10 Grover: oracle, diffusion, amplitude amplification, $O(\sqrt{N})$.
- 11 Deutsch-Jozsa: constant vs balanced, phase kickback, all-zero rule.
- 12 Ripple adder: sum/carry equations, CNOT=XOR, Toffoli=AND/carry.

16. Final coverage checklist

Source file	Slides/pages	Coverage status	Inline diagrams inserted from
_000_Quantum AGI Computing-Course Overview.pptx	9 slides	Covered course framing, superposition/entanglement, AGI/agent/blockchain context. Reference screenshots inspected.	slide 5
_001_Principles and Roots of Quantum Computing.pptx	22 slides	Covered quantization, uncertainty, coherence, wavefunction, superposition, entanglement, interference, wave behavior, timeline, and hardware examples.	slides 4, 10, 14, 21, 22
_002_Quantum Gates and Operators Jan 2026-updated.pptx	40 slides	Covered gate/operator motivation, classical vs quantum circuits, Bloch sphere, H/X/Y/Z/I/phase/S/T/U gates, SWAP, controlled gates, CX/CY/CZ, Toffoli, Fredkin, simulator.	slides 3, 7, 8, 11, 13, 16-18, 20, 24, 25, 27-30, 32-34, 37, 38, 40
_003_Entanglement.-Bell States.pptx	24 slides	Covered multi-qubit initialization, tensor products, CNOT, entanglement implementation, Bell states, Bell circuits, and corrected CNOT slide error.	slides 4, 6-11, 13-15, 17, 19, 21-23
_004_Quantum Teleportation-2026-updated.pptx	12 slides	Covered teleportation definition, setup, Bell resource, protocol, correction table, and algebraic example.	slides 5-7, 9-12
_005_Shor Integer Factorization Algorithm.pptx	11 slides	Covered factoring motivation, period/order, Shor workflow, N=15 example, QFT role and plots.	slides 3, 5-11
_006_The Quantum Fourier Transform (QFT).pptx	10 slides	Covered DFT, QFT basis/general formulas, circuit components, full circuit, complexity, one- and two-qubit examples.	slides 3-10
_007_Grover#U2019s Algorithm-Jan 2026.pptx	18 slides	Covered unstructured search, steps, oracle, performance graph, diffusion, reflection, amplitude amplification. Slide 18 was visually blank/low-content and has no separate concept.	slides 3-5, 7, 9, 10, 13-16
_008_Deutsch-Jozsa Algorithm-Revisited.pptx	16 slides	Covered constant/balanced promise, circuits, bundled notation, phase kickback, oracle role, measurement rule, histograms.	slides 3, 5-9, 11, 13-16
_009_Ripple Adder -Quantum.pptx	22 slides	Covered ripple-carry definition, gate roles, equations, qubit allocation, circuit diagram, code structure, simulation, results, circuit analysis.	slides 3-6, 8-12, 14-22
_010_Generative AI.pptx	27 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_011_Getting Started With LangChain.pptx	17 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_012_MLP-How it Works -(MultiLayer Perceptrons MLP) Example.pptx	23 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_013_The Context-Machine Learning - Overview.pptx	17 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_014_The Agentic AI Revolution.pptx	20 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_015_Demo -Cognitive Loop-LangChain Comprehensive Example-2 .docx	31 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_016_LangChain Multiagent.docx	8 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_017_Autogen Single Agent Formatted.docx	10 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_018_AutoGen MultiAgent.docx	10 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope

Source file	Slides/pages	Coverage status	Inline diagrams inserted from
_019_Crew Single Agent Formatted.docx	14 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_020_CrewAI MultiAgent.docx	20 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_021_Blockchain Overview.pptx	26 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_022_Bitcoin Proof of Work Algorithm.pptx	13 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_023_How Bitcoin Works.pptx	11 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_024_Ethereum How It Works -1.pptx	32 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_025_Solidity By Example1.docx	9 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_026_Ethereum Smart Contracts Practice Sheet.docx	6 pages	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_027Ethereum Proof of Stake.pptx	19 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_028_Solana - Fastest Blockchain--Prroof of History .pptx	11 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope
_029_PolkaDot - an Emerging Blockchain Systems.pptx	11 slides	Inspected and counted; outside the requested Quantum Computing topic, so not summarized in this guide.	None - out of scope

17. Verification notes

- All files in the zip were listed and counted.
- All 10 relevant Quantum Computing slide decks were converted to PDF and rendered to slide images for visual checking.
- Important diagrams, circuits, tables, code screenshots, workflow diagrams, charts, and examples were inserted inline in the corresponding topic sections.
- The obvious CNOT text error in _003 slide 6 was corrected and explicitly marked.
- Some reference-only screenshots in the course overview were inspected but not reproduced because they do not add exam content beyond the notes.
- The guide is study-optimized: it explains concepts and inserts key visuals; it is not a page-by-page archive of every slide screenshot.

Exam-important

Final status: every Quantum Computing source file _000 through _009 is covered. Important diagrams are inserted inline where they belong. Non-quantum files _010 through _029 were inspected and intentionally excluded from the summary content.